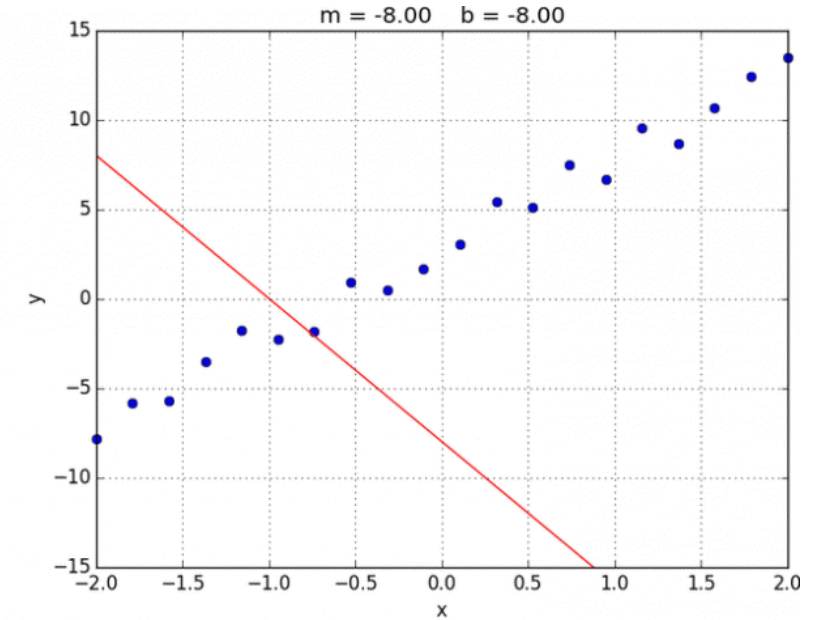
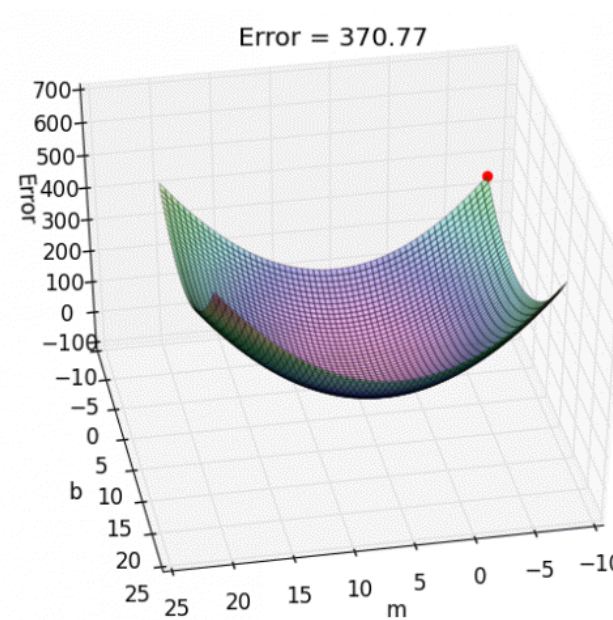


Deep Learning

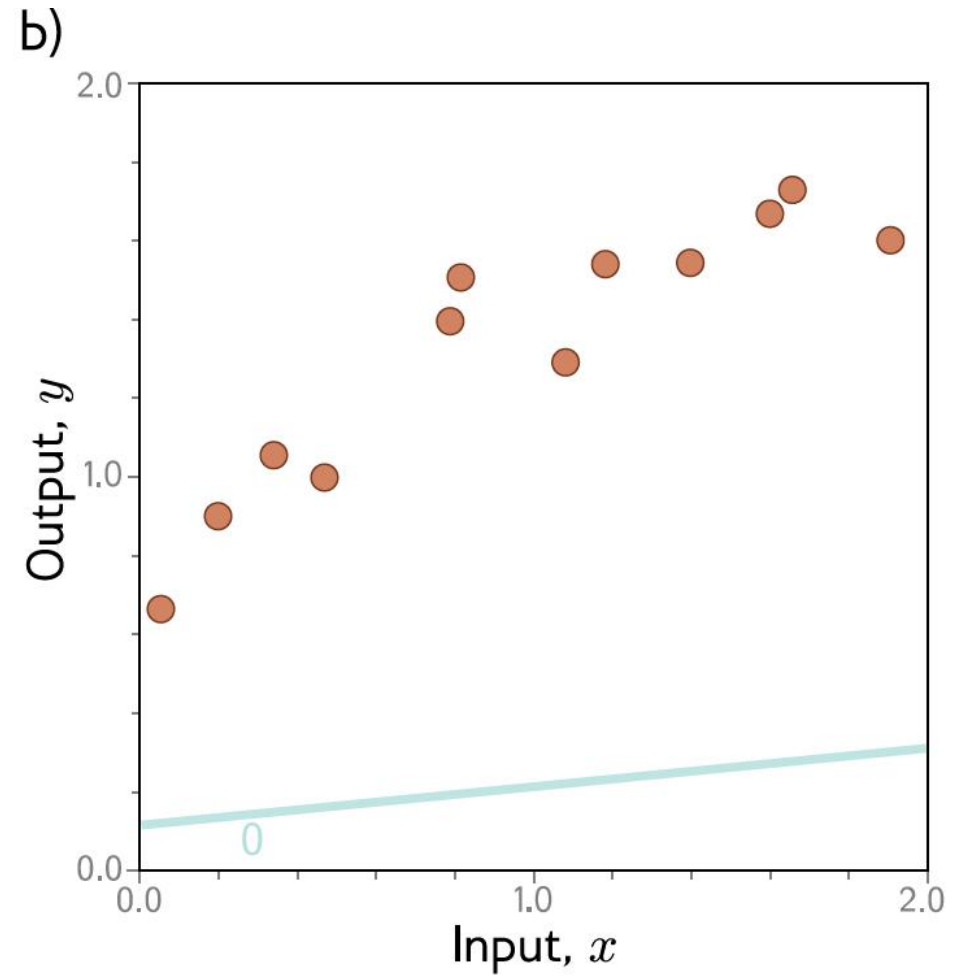
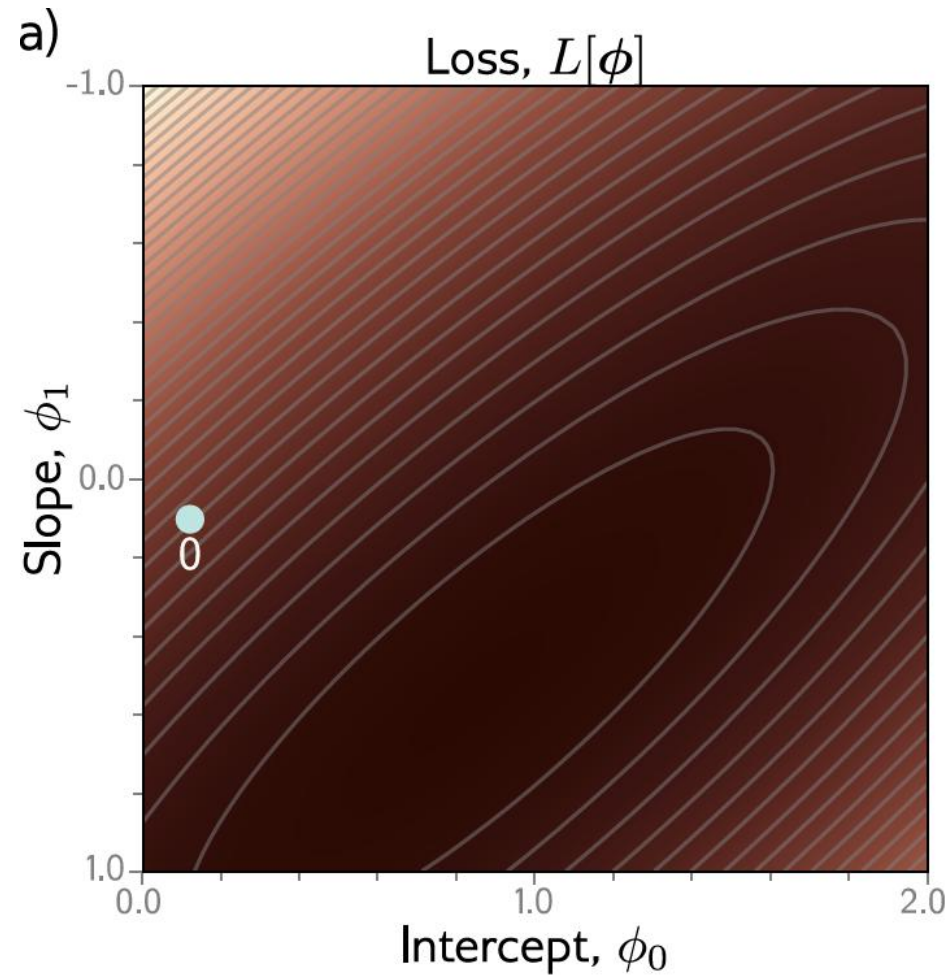


LECTURE 5B – OPTIMIZATION, GENERALIZATION, REGULARIZATION

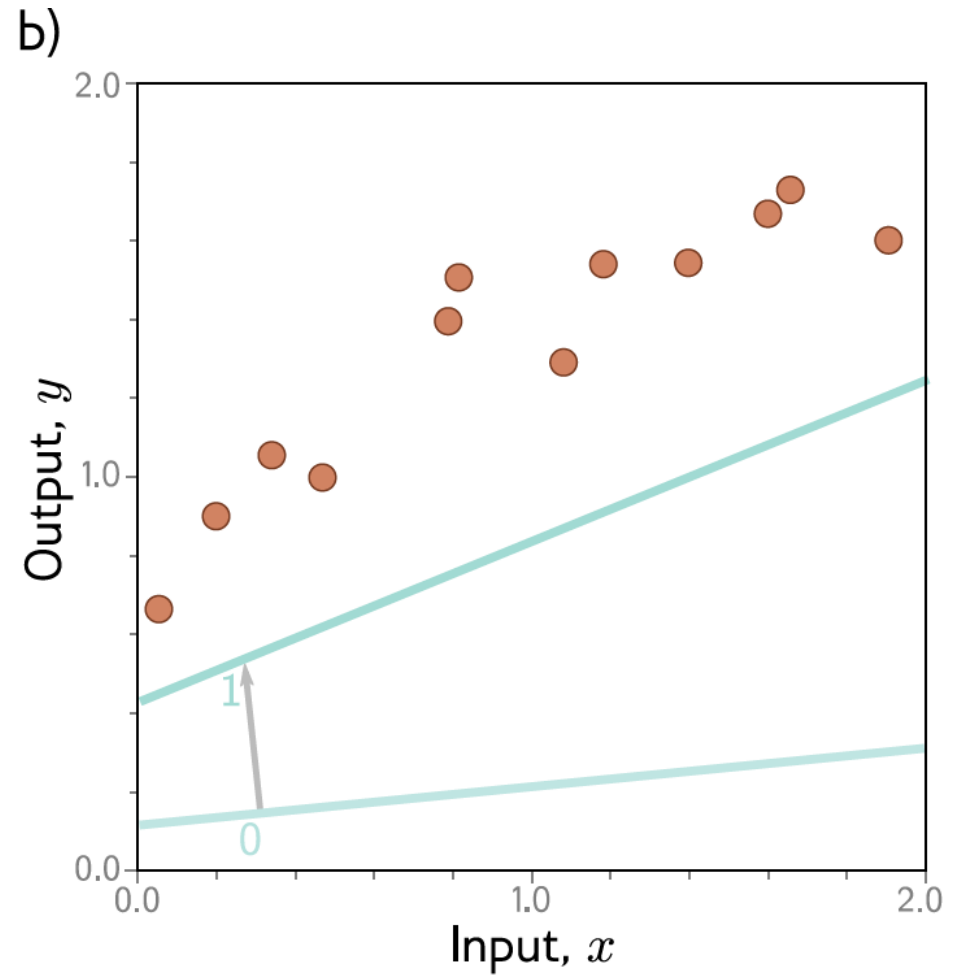
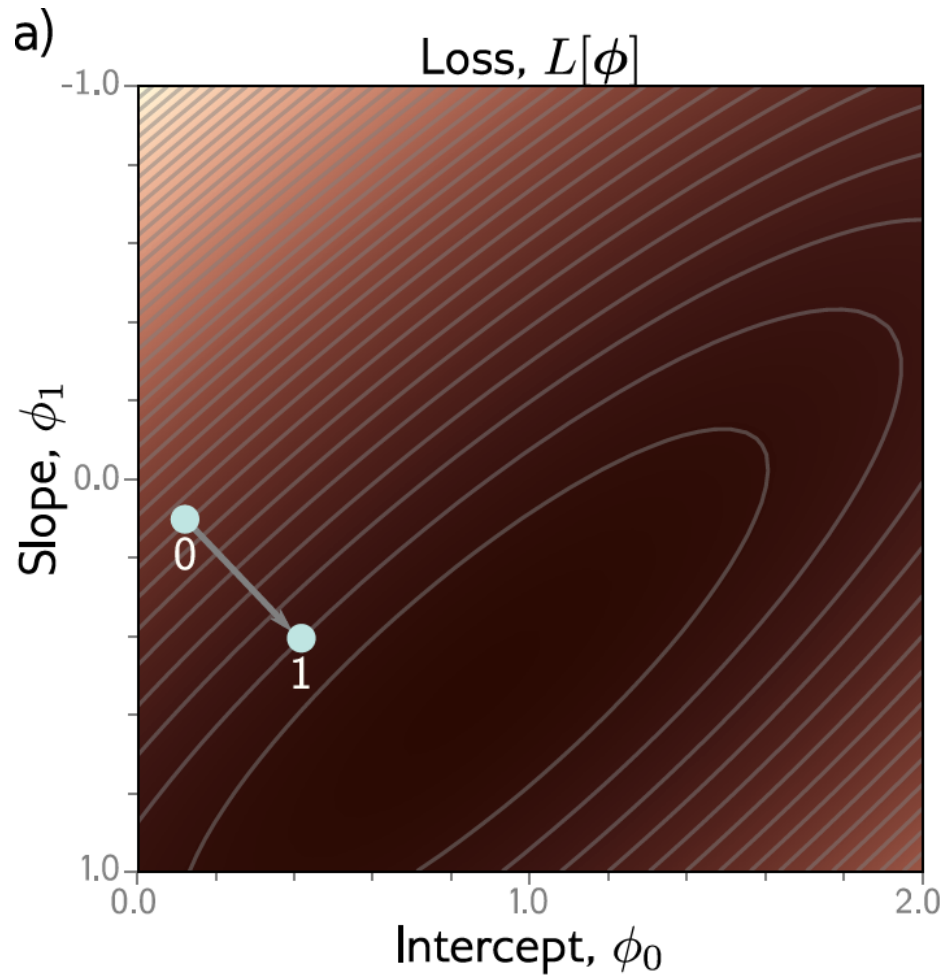
Kevin Bryson

<https://alykhantejani.github.io/a-brief-introduction-to-gradient-descent/>

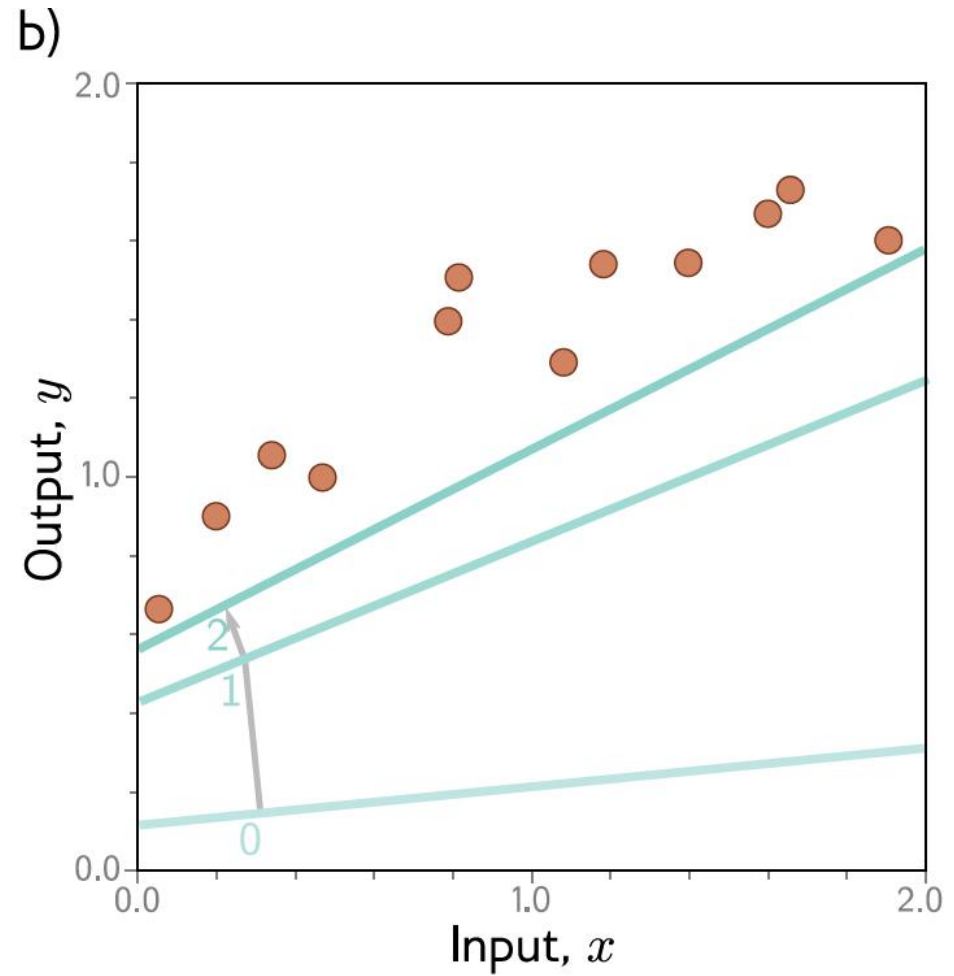
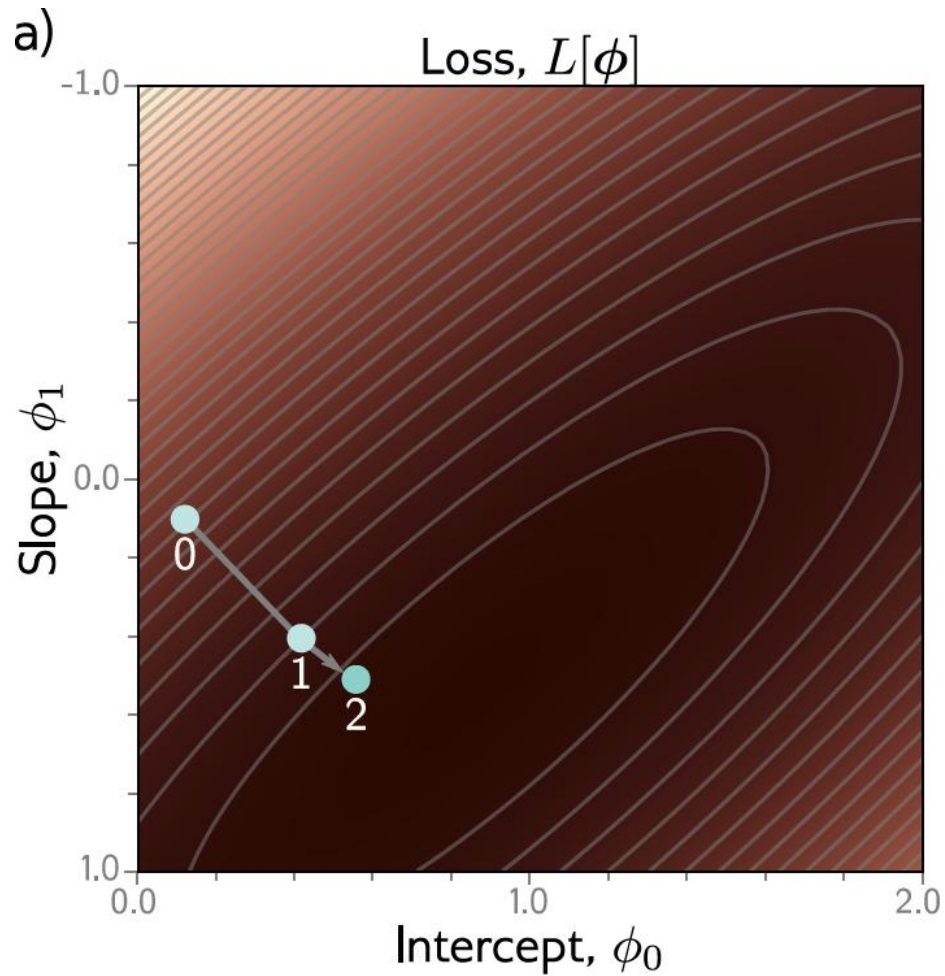
Example: 1D Linear regression training



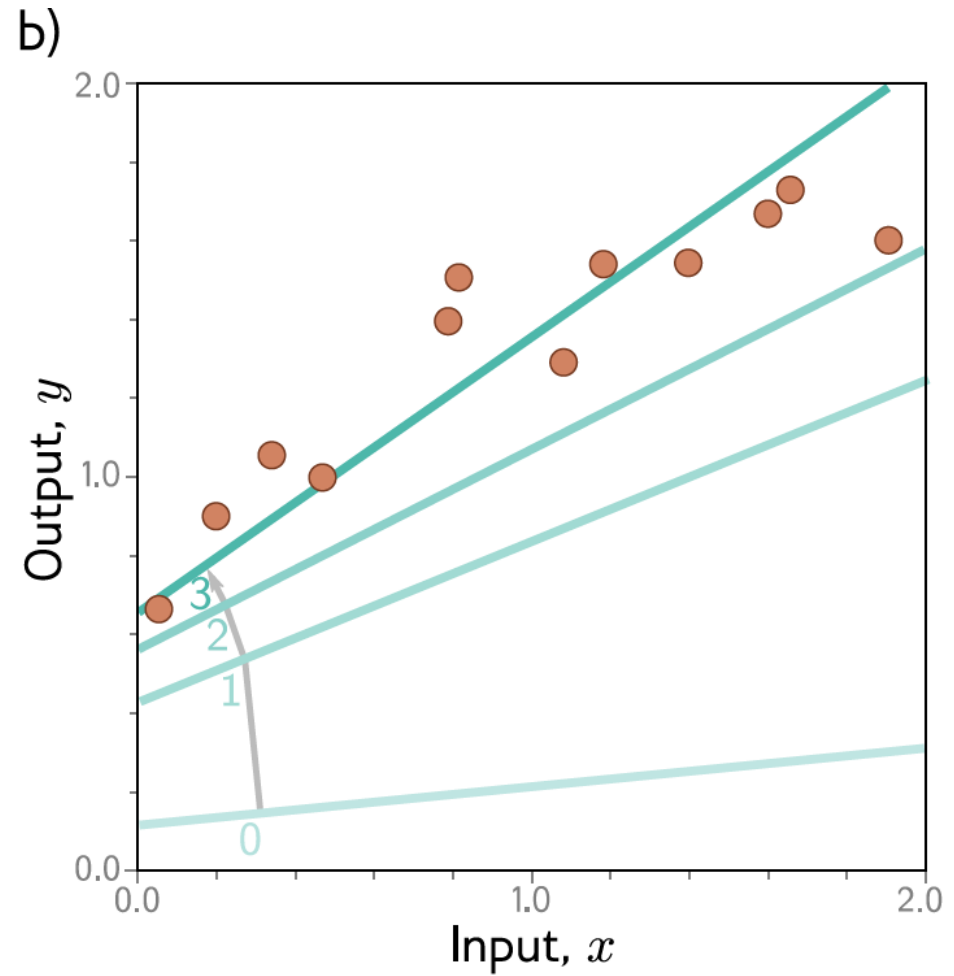
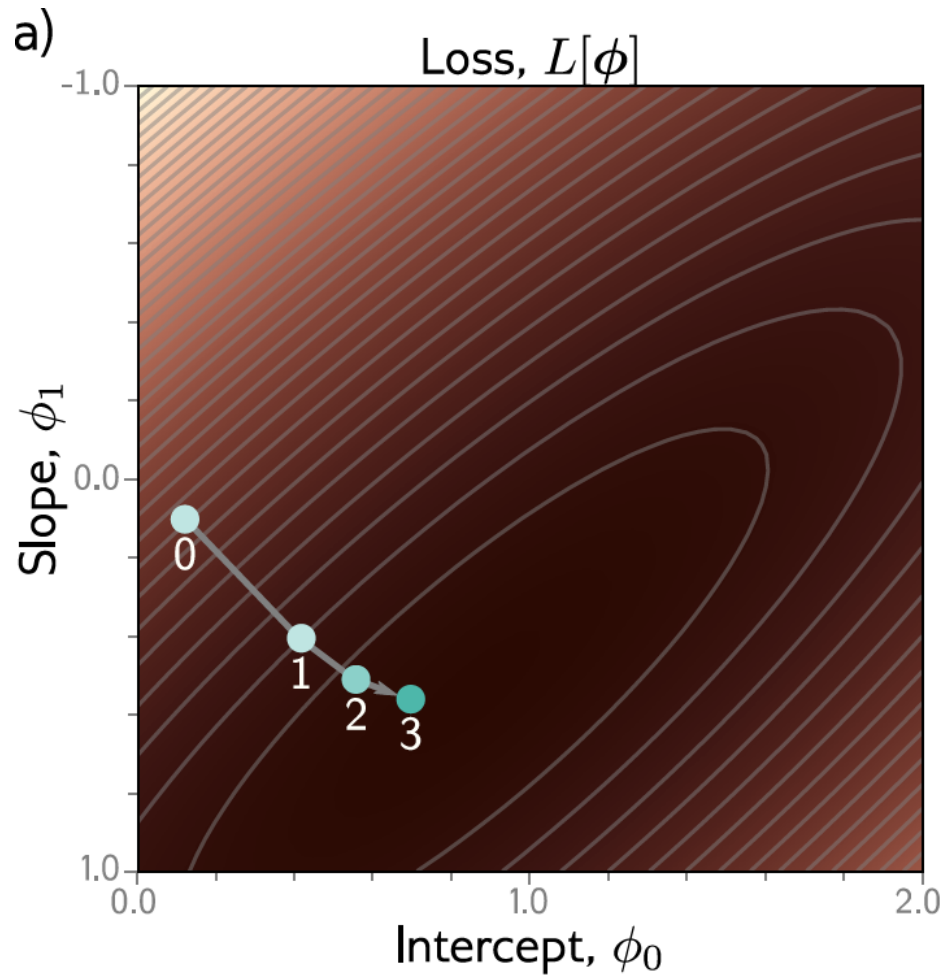
Example: 1D Linear regression training



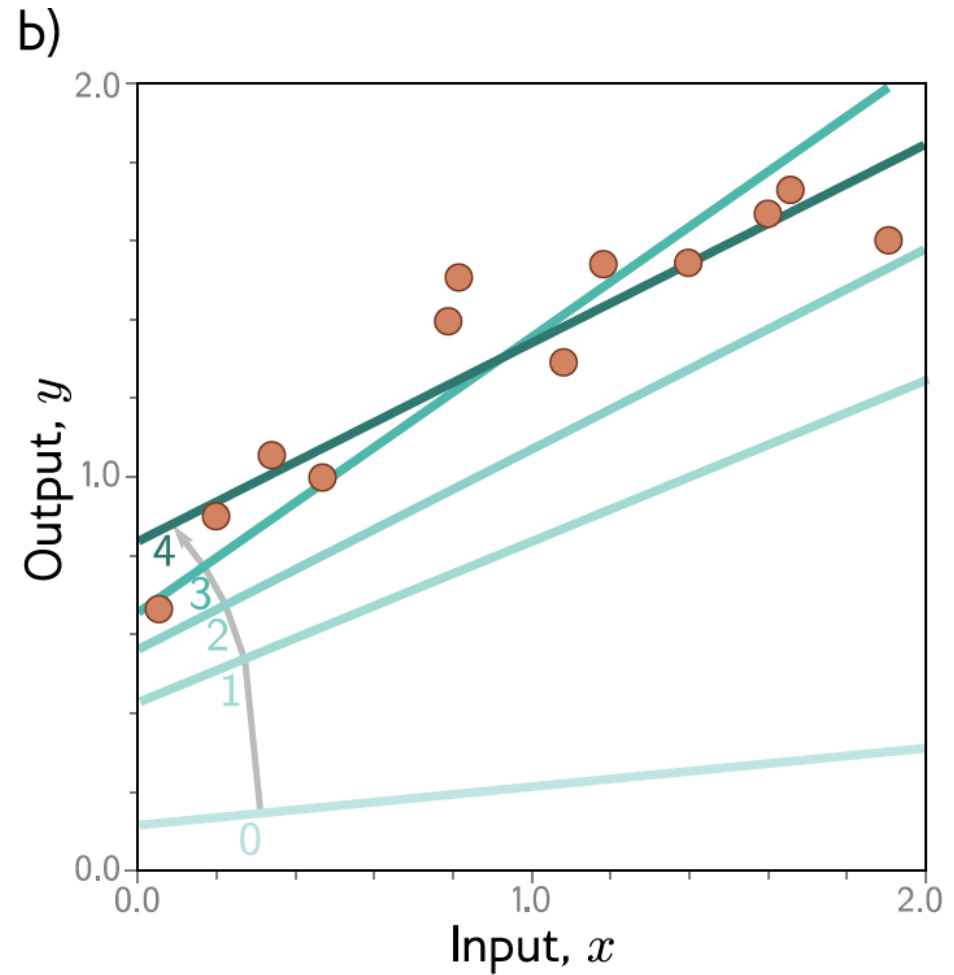
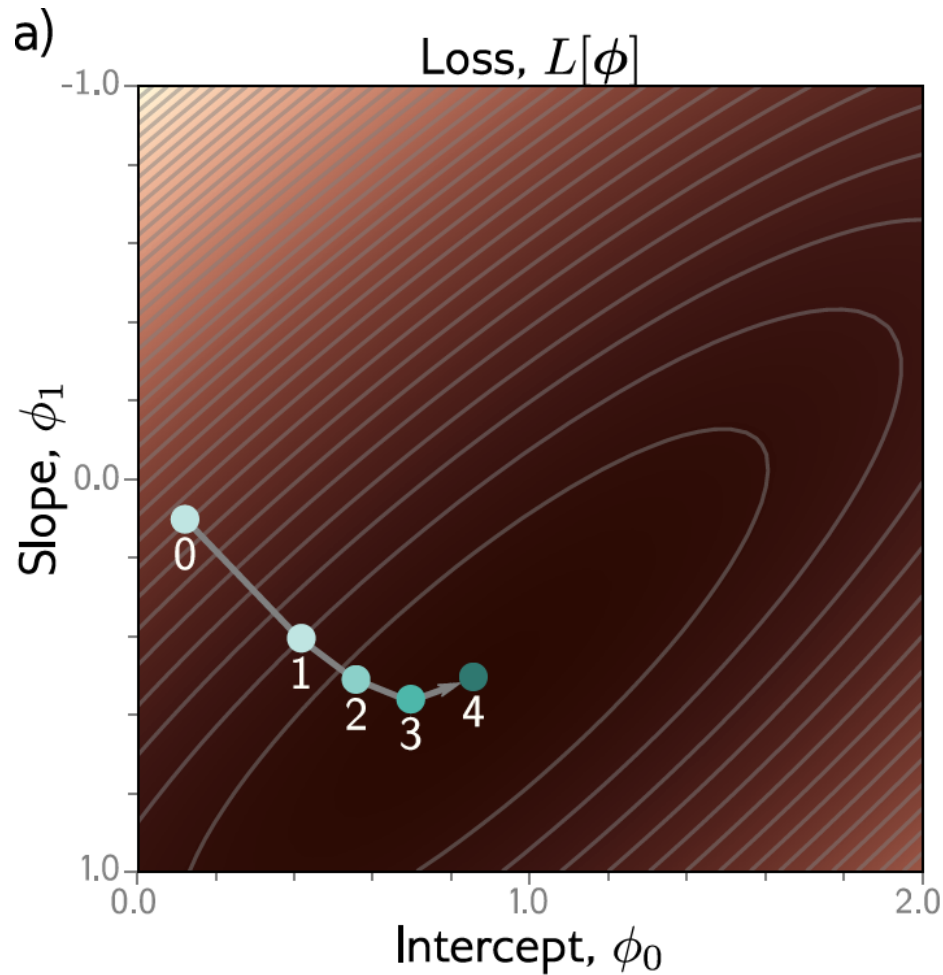
Example: 1D Linear regression training



Example: 1D Linear regression training

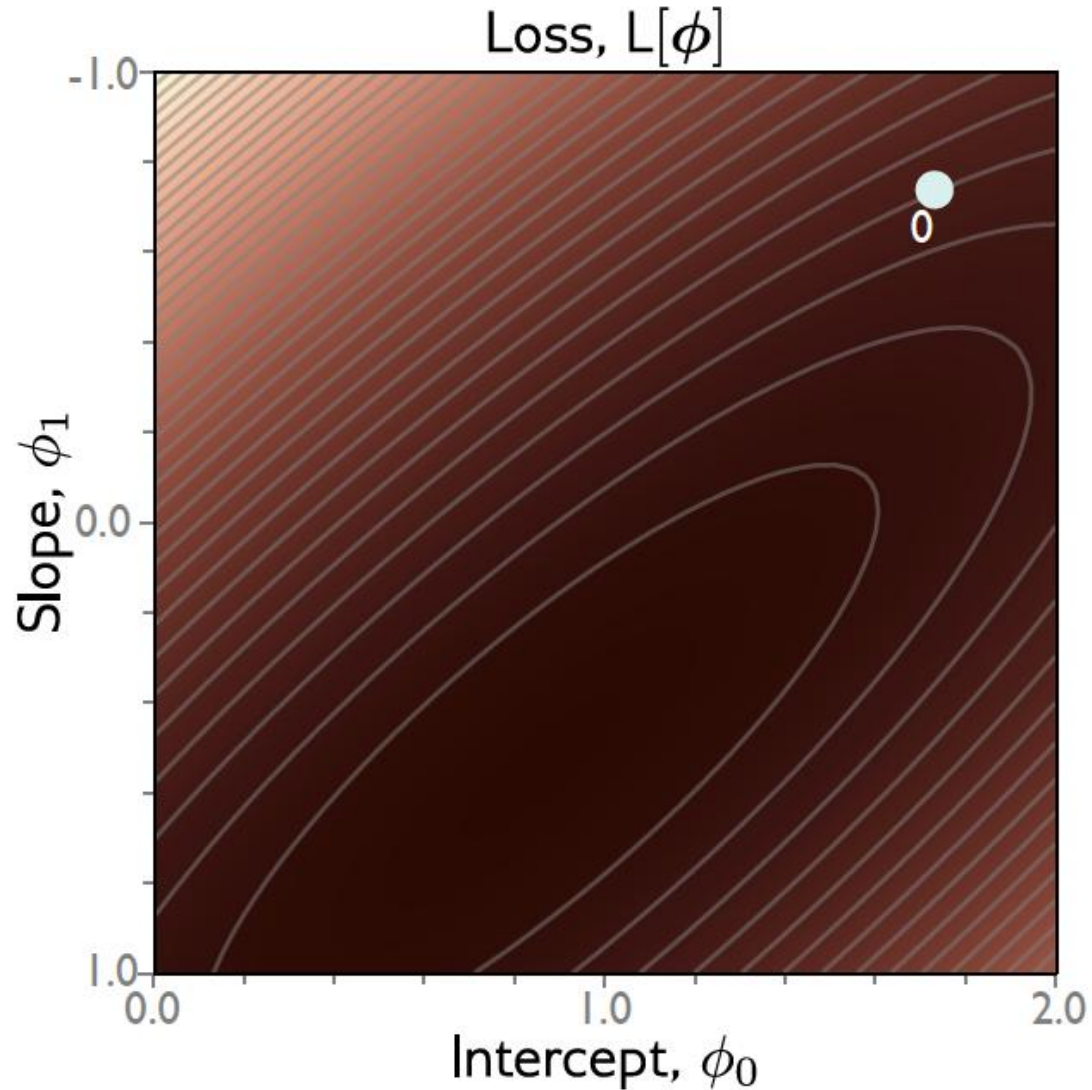


Example: 1D Linear regression training



This technique is known as **gradient descent**

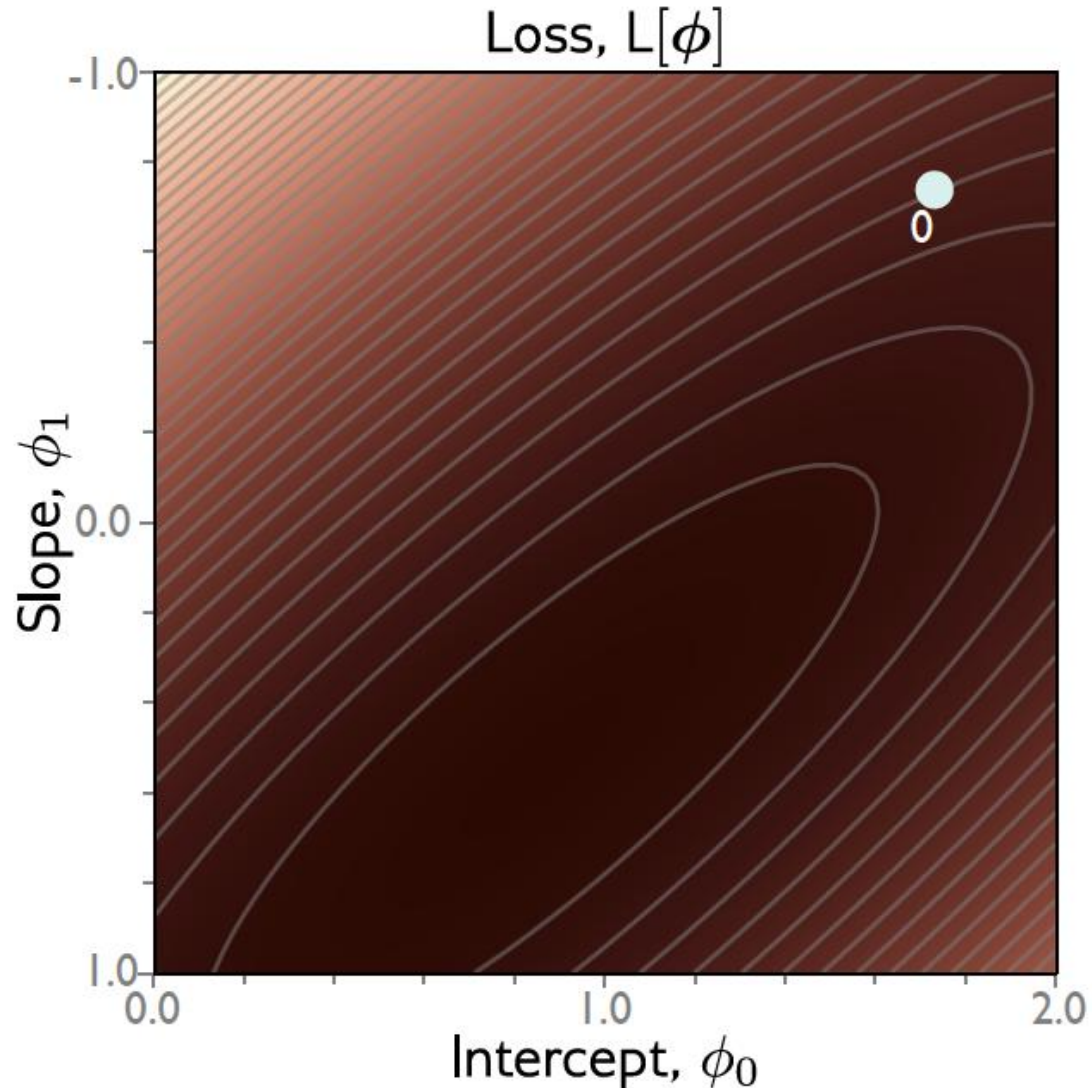
Gradient descent



Step 1: Compute derivatives (slopes of function) with Respect to the parameters

$$\begin{aligned} L[\phi] &= \sum_{i=1}^I \ell_i = \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \end{aligned}$$

Gradient descent

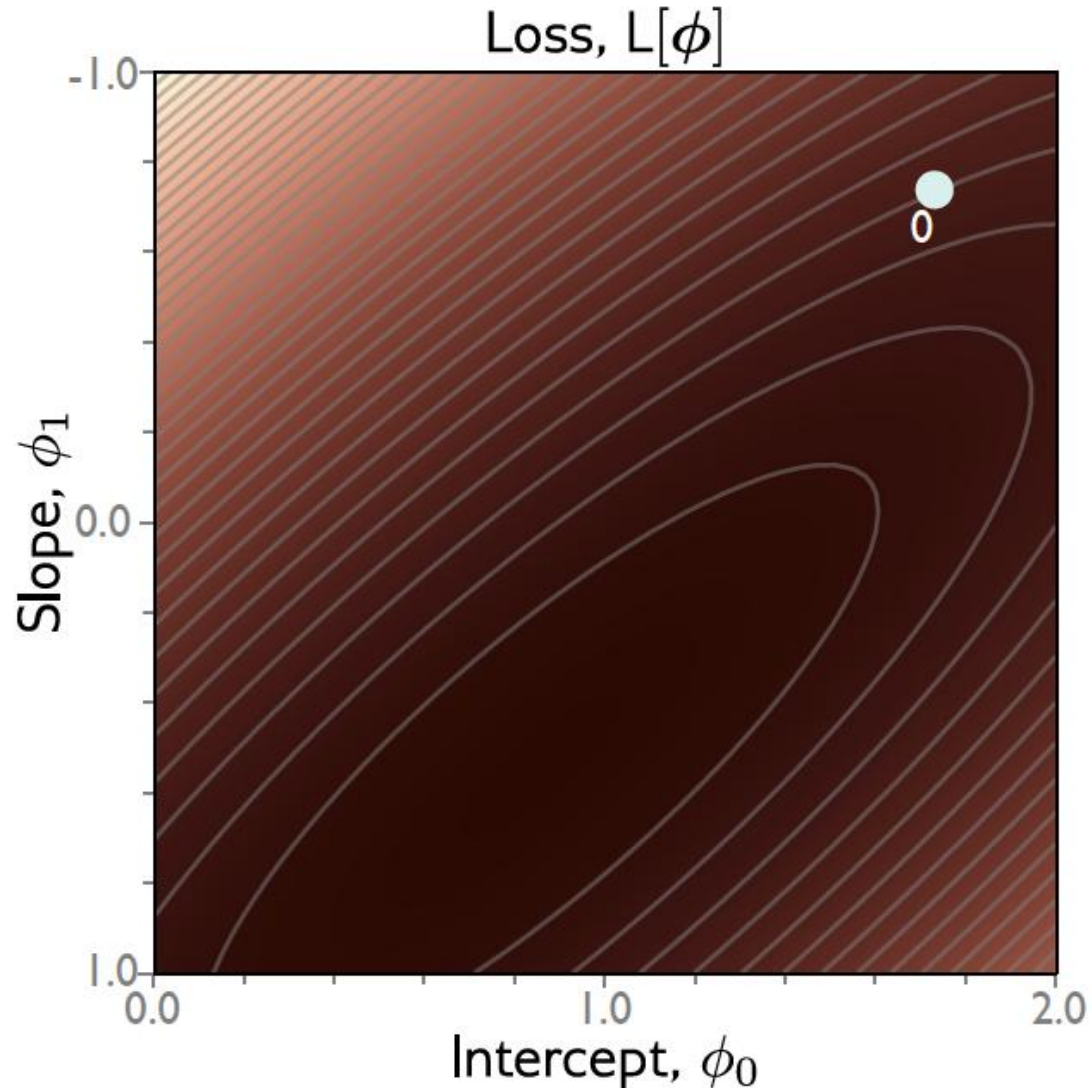


Step 1: Compute derivatives (slopes of function) with Respect to the parameters

$$\begin{aligned} L[\phi] &= \sum_{i=1}^I \ell_i = \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \end{aligned}$$

$$\frac{\partial L}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^I \ell_i = \sum_{i=1}^I \frac{\partial \ell_i}{\partial \phi}$$

Gradient descent



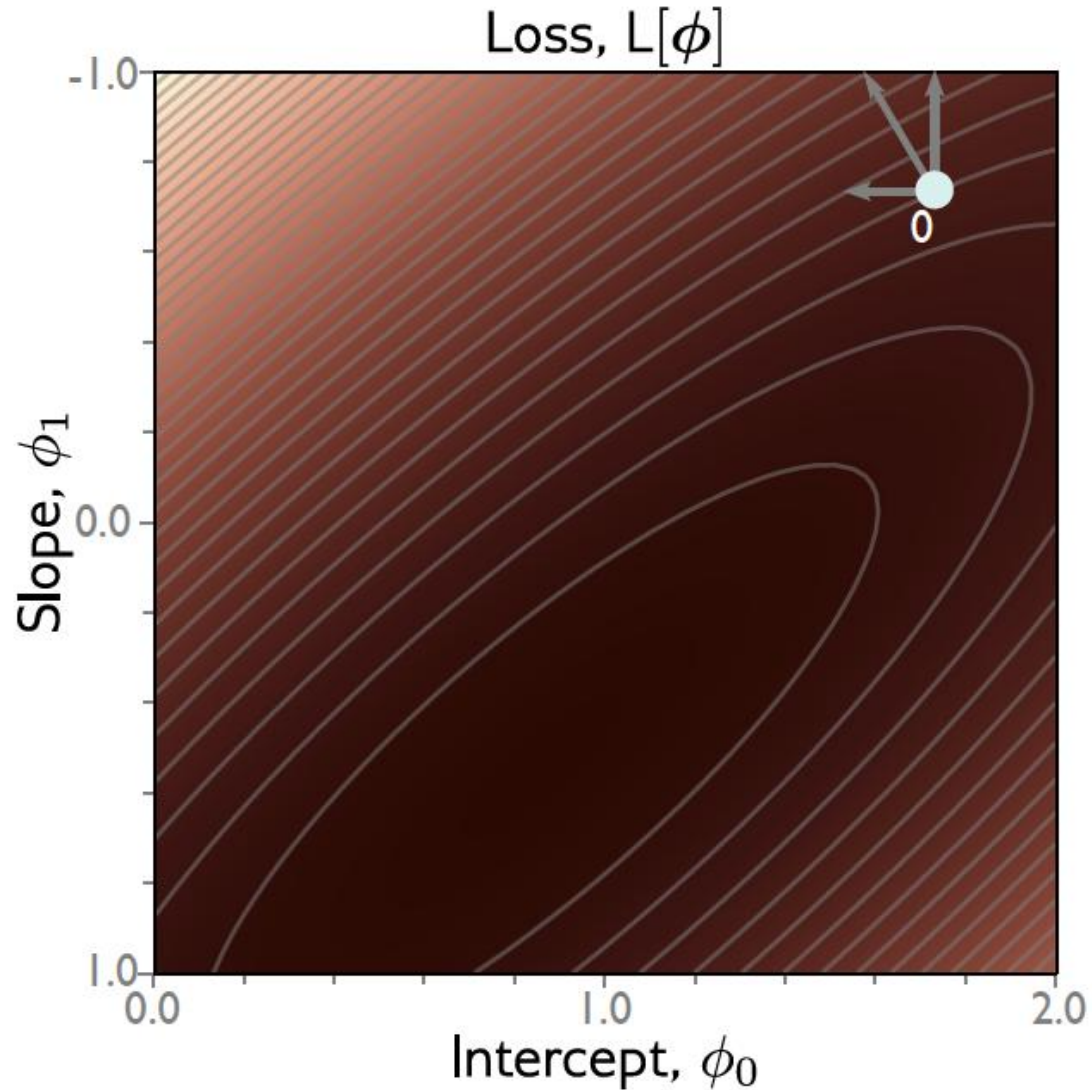
Step 1: Compute derivatives (slopes of function) with Respect to the parameters

$$\begin{aligned} L[\phi] &= \sum_{i=1}^I \ell_i = \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \end{aligned}$$

$$\frac{\partial L}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^I \ell_i = \sum_{i=1}^I \frac{\partial \ell_i}{\partial \phi}$$

$$\frac{\partial \ell_i}{\partial \phi} = \begin{bmatrix} \frac{\partial \ell_i}{\partial \phi_0} \\ \frac{\partial \ell_i}{\partial \phi_1} \end{bmatrix} = \begin{bmatrix} 2(\phi_0 + \phi_1 x_i - y_i) \\ 2x_i(\phi_0 + \phi_1 x_i - y_i) \end{bmatrix}$$

Gradient descent

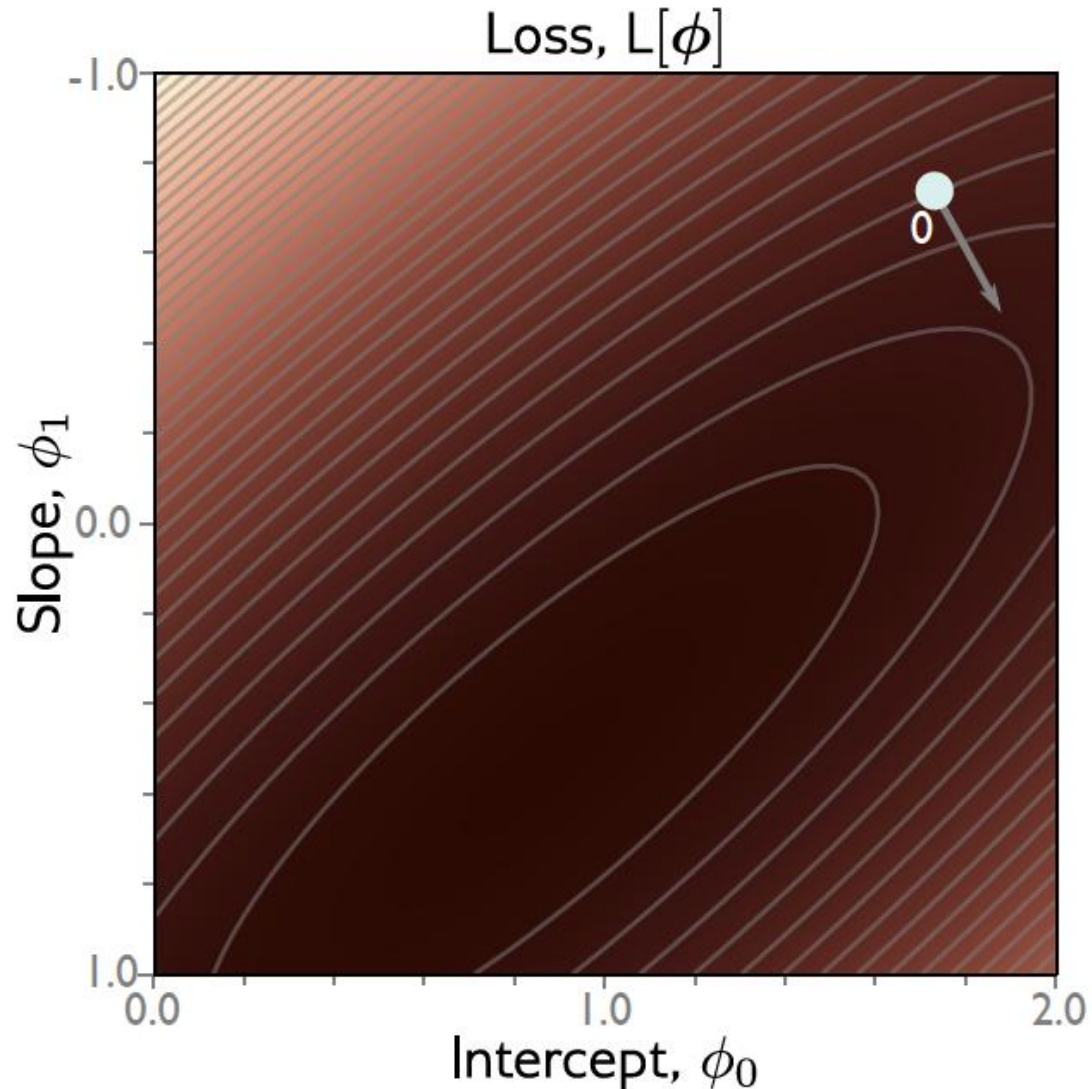


Step 1: Compute derivatives (slopes of function) with Respect to the parameters

$$\frac{\partial L}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^I \ell_i = \sum_{i=1}^I \frac{\partial \ell_i}{\partial \phi}$$

$$\frac{\partial \ell_i}{\partial \phi} = \begin{bmatrix} \frac{\partial \ell_i}{\partial \phi_0} \\ \frac{\partial \ell_i}{\partial \phi_1} \end{bmatrix} = \begin{bmatrix} 2(\phi_0 + \phi_1 x_i - y_i) \\ 2x_i(\phi_0 + \phi_1 x_i - y_i) \end{bmatrix}$$

Gradient descent



Step 1: Compute derivatives (slopes of function) with Respect to the parameters

$$\frac{\partial L}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^I \ell_i = \sum_{i=1}^I \frac{\partial \ell_i}{\partial \phi}$$

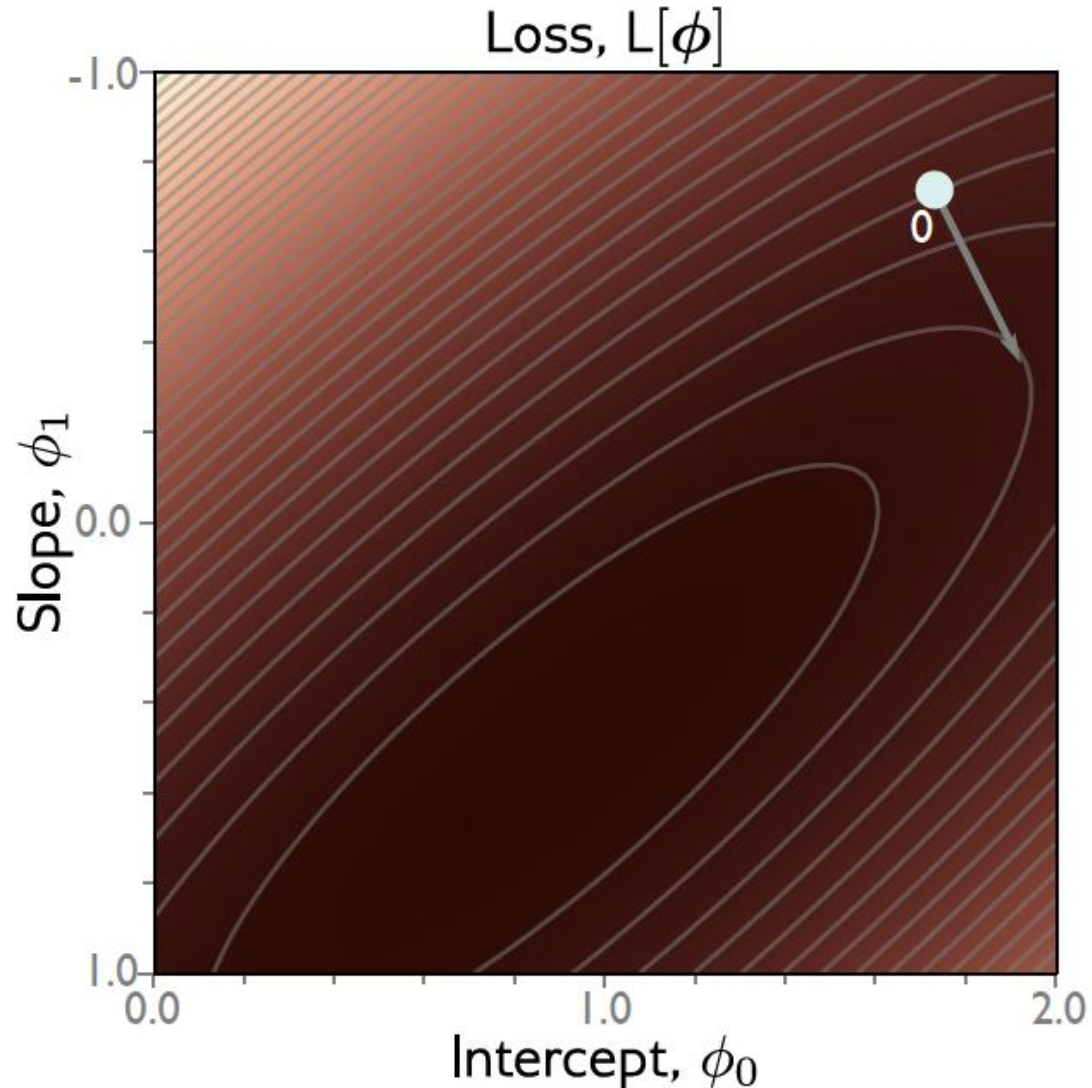
$$\frac{\partial \ell_i}{\partial \phi} = \begin{bmatrix} \frac{\partial \ell_i}{\partial \phi_0} \\ \frac{\partial \ell_i}{\partial \phi_1} \end{bmatrix} = \begin{bmatrix} 2(\phi_0 + \phi_1 x_i - y_i) \\ 2x_i(\phi_0 + \phi_1 x_i - y_i) \end{bmatrix}$$

Step 2: Update parameters according to rule

$$\phi \longleftarrow \phi - \alpha \frac{\partial L}{\partial \phi}$$

α = step size or **learning rate** if fixed

Gradient descent



Step 1: Compute derivatives (slopes of function) with Respect to the parameters

$$\frac{\partial L}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^I \ell_i = \sum_{i=1}^I \frac{\partial \ell_i}{\partial \phi}$$

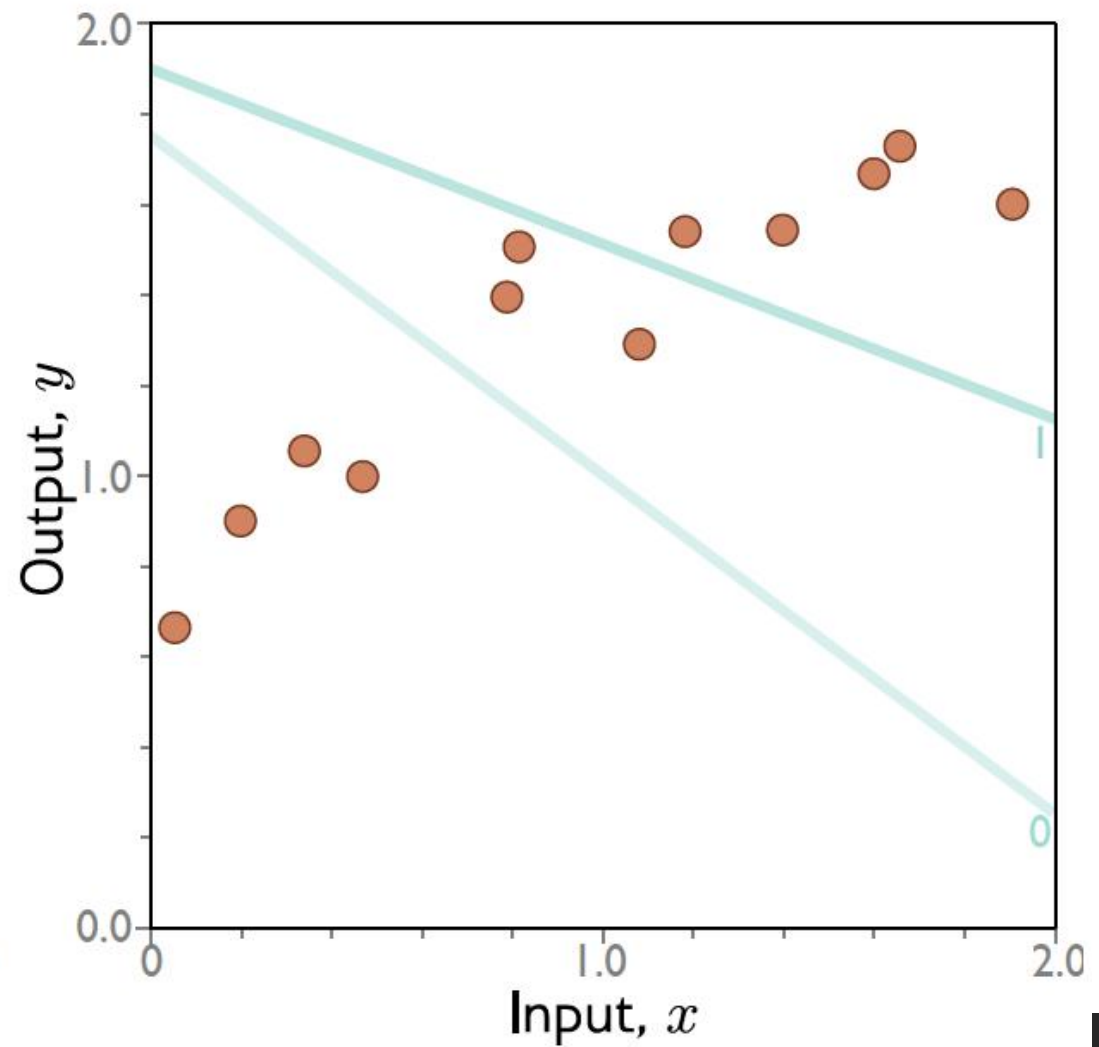
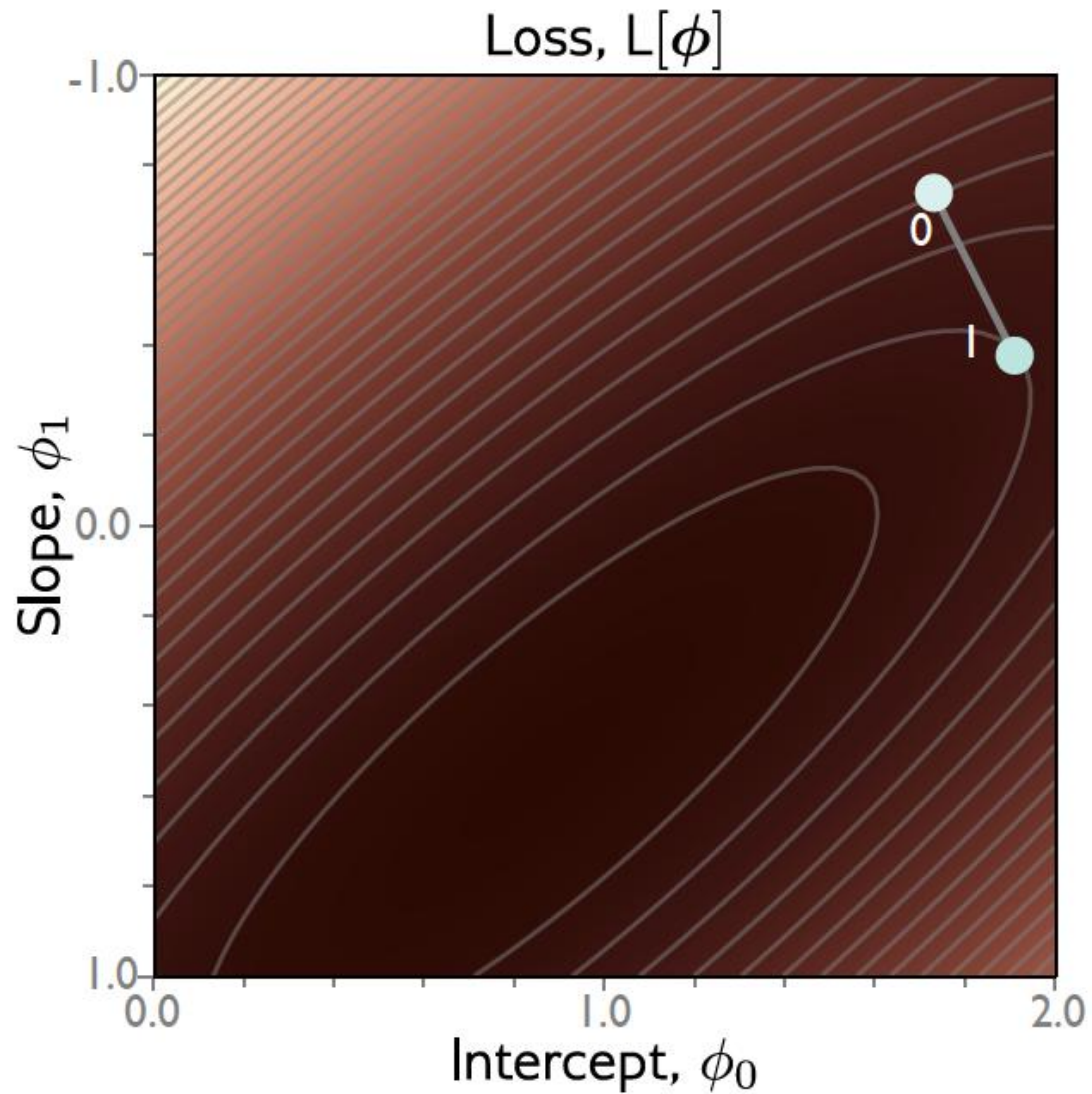
$$\frac{\partial \ell_i}{\partial \phi} = \begin{bmatrix} \frac{\partial \ell_i}{\partial \phi_0} \\ \frac{\partial \ell_i}{\partial \phi_1} \end{bmatrix} = \begin{bmatrix} 2(\phi_0 + \phi_1 x_i - y_i) \\ 2x_i(\phi_0 + \phi_1 x_i - y_i) \end{bmatrix}$$

Step 2: Update parameters according to rule

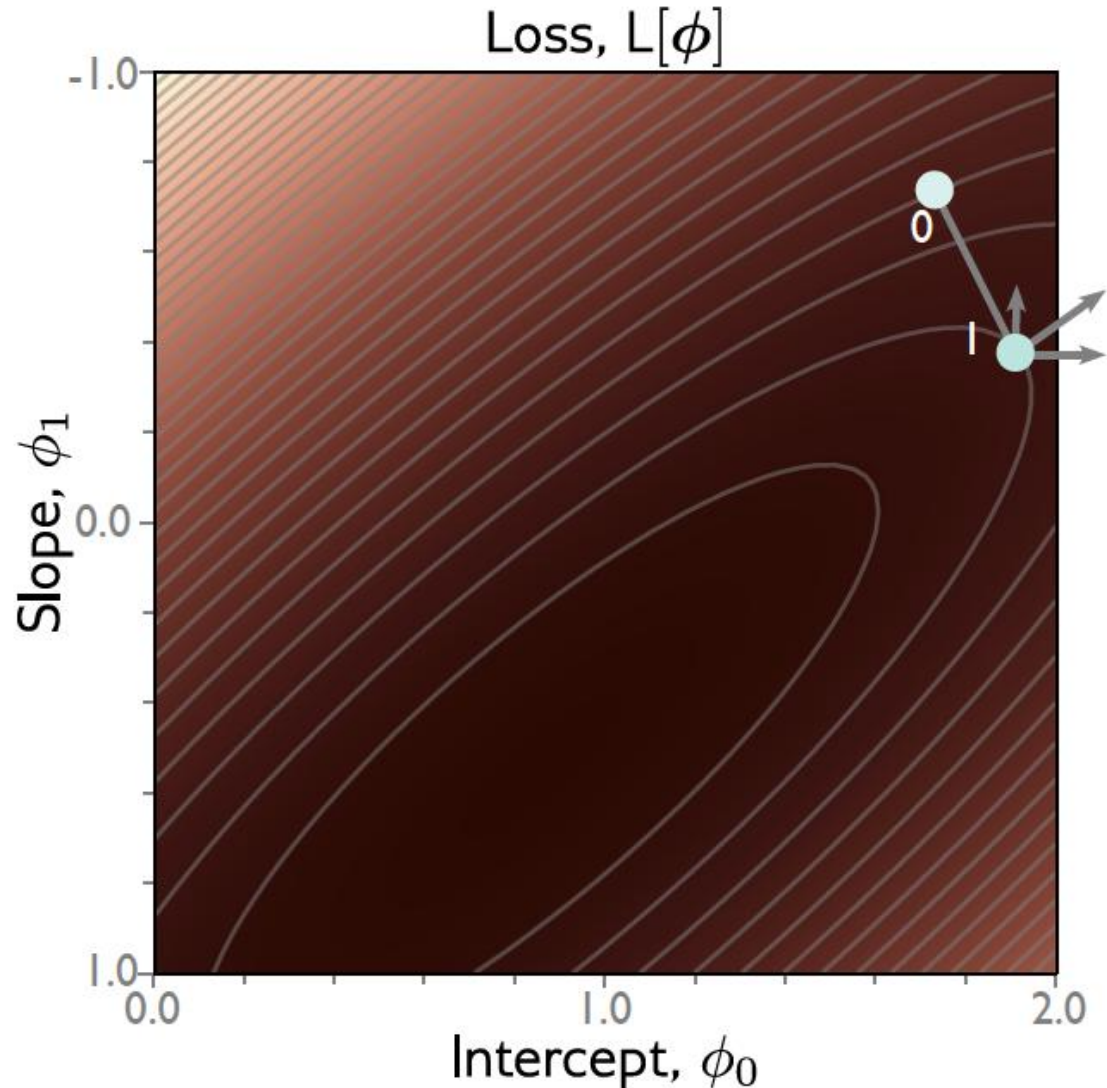
$$\phi \longleftarrow \phi - \alpha \frac{\partial L}{\partial \phi}$$

α = step size

Gradient descent



Gradient descent



Step 1: Compute derivatives (slopes of function) with Respect to the parameters

$$\frac{\partial L}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^I \ell_i = \sum_{i=1}^I \frac{\partial \ell_i}{\partial \phi}$$

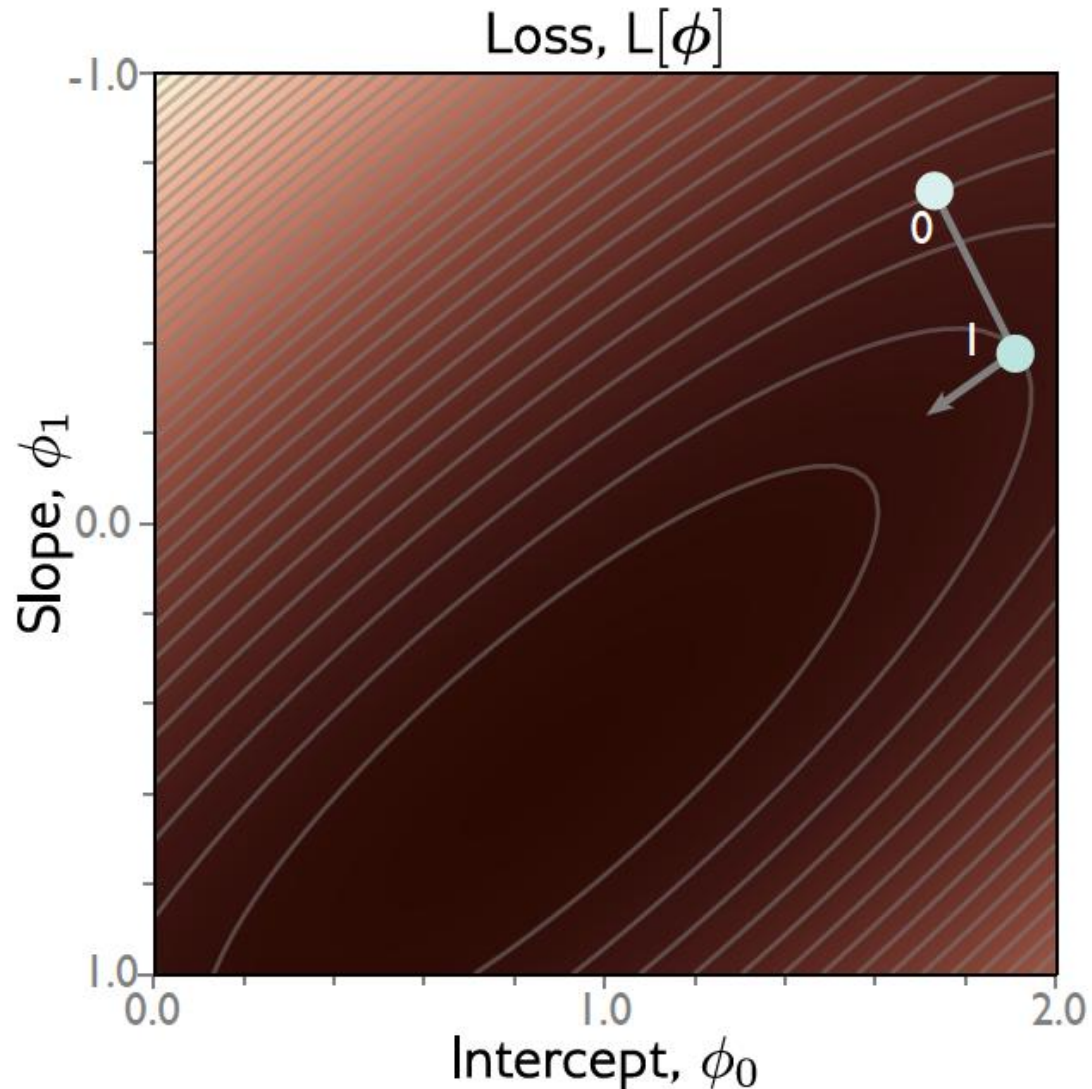
$$\frac{\partial \ell_i}{\partial \phi} = \begin{bmatrix} \frac{\partial \ell_i}{\partial \phi_0} \\ \frac{\partial \ell_i}{\partial \phi_1} \end{bmatrix} = \begin{bmatrix} 2(\phi_0 + \phi_1 x_i - y_i) \\ 2x_i(\phi_0 + \phi_1 x_i - y_i) \end{bmatrix}$$

Step 2: Update parameters according to rule

$$\phi \longleftarrow \phi - \alpha \frac{\partial L}{\partial \phi}$$

α = step size

Gradient descent



Step 1: Compute derivatives (slopes of function) with Respect to the parameters

$$\frac{\partial L}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^I \ell_i = \sum_{i=1}^I \frac{\partial \ell_i}{\partial \phi}$$

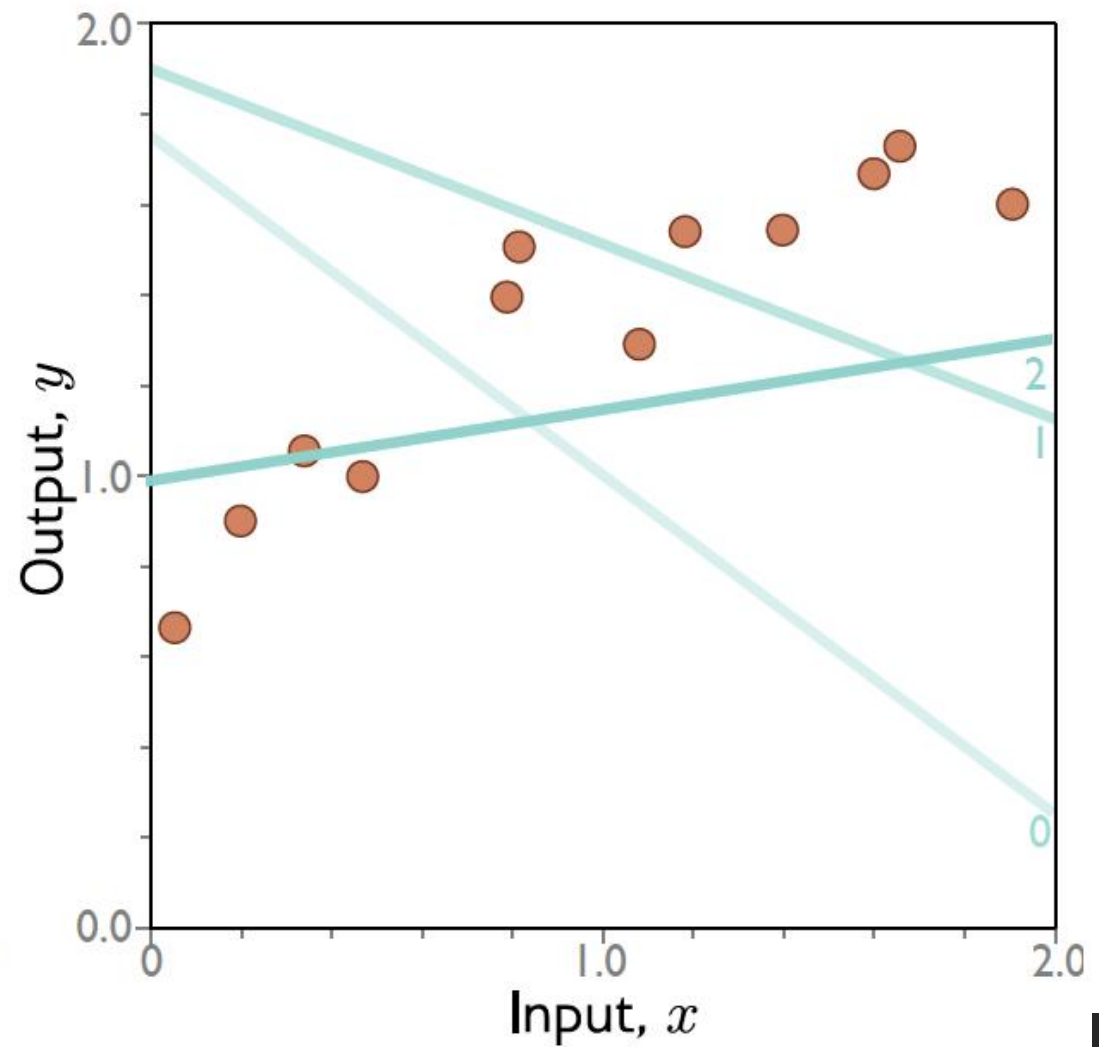
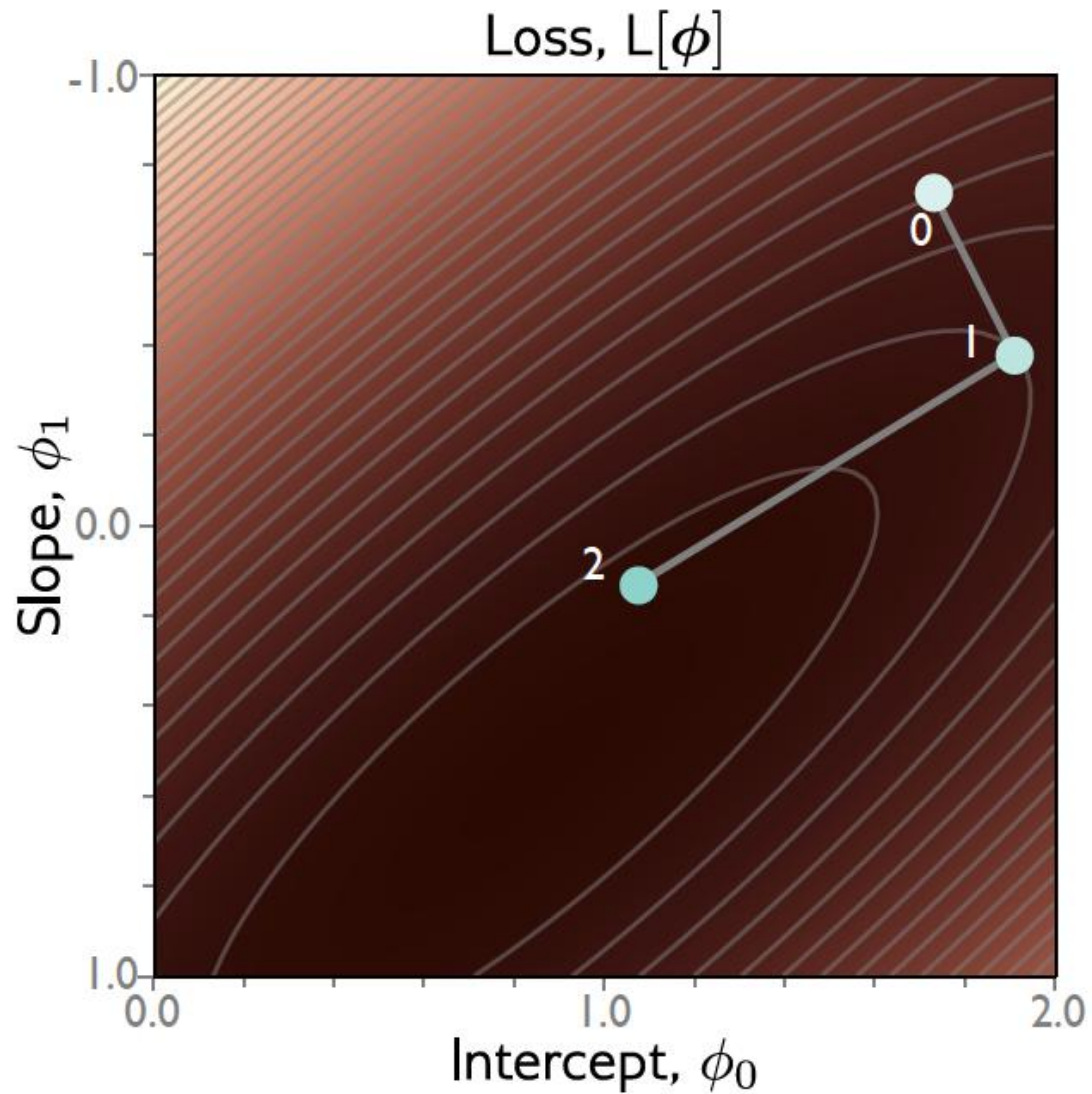
$$\frac{\partial \ell_i}{\partial \phi} = \begin{bmatrix} \frac{\partial \ell_i}{\partial \phi_0} \\ \frac{\partial \ell_i}{\partial \phi_1} \end{bmatrix} = \begin{bmatrix} 2(\phi_0 + \phi_1 x_i - y_i) \\ 2x_i(\phi_0 + \phi_1 x_i - y_i) \end{bmatrix}$$

Step 2: Update parameters according to rule

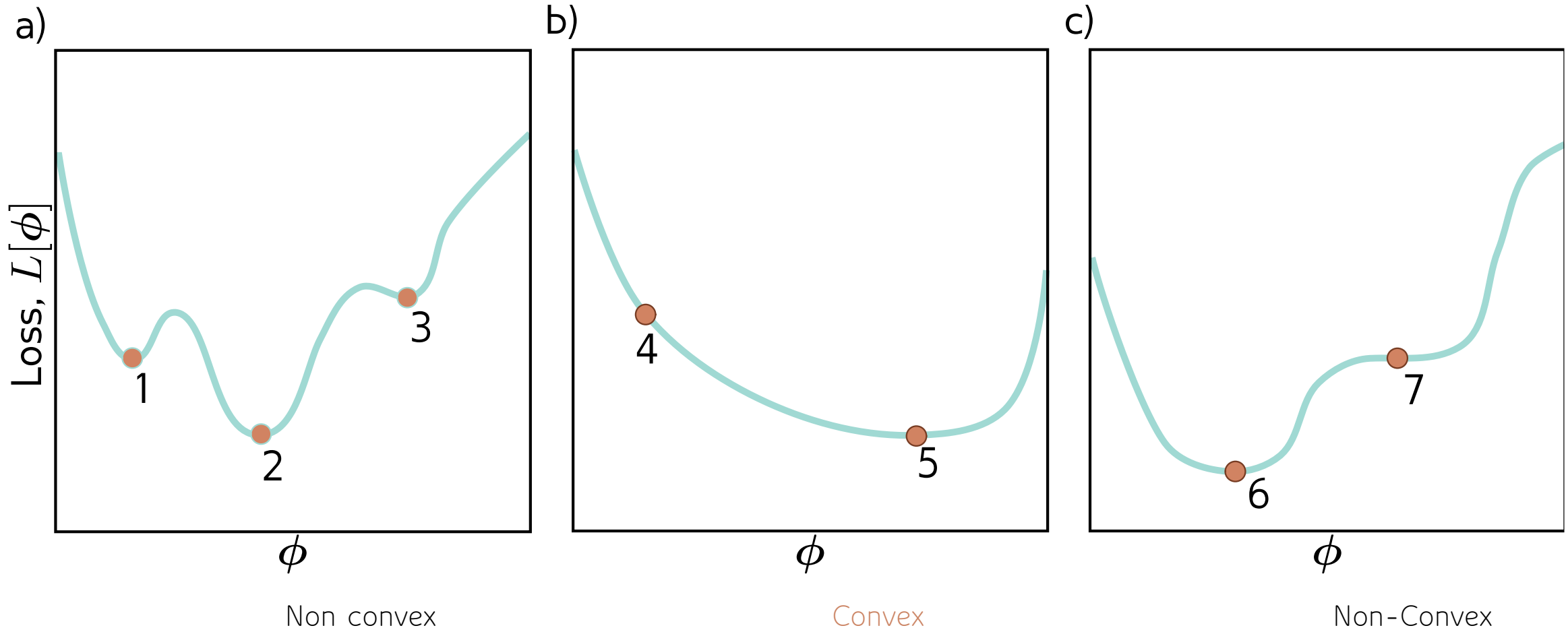
$$\phi \leftarrow \phi - \alpha \frac{\partial L}{\partial \phi}$$

α = step size

Gradient descent

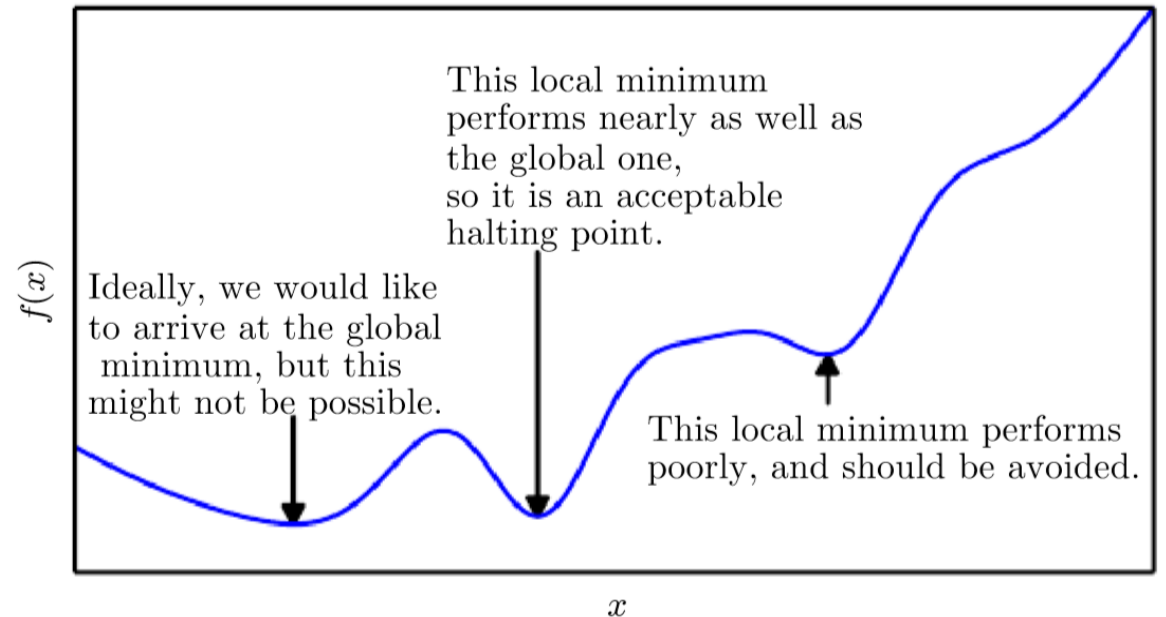


Deep Learning does not usually have a convex loss function ...



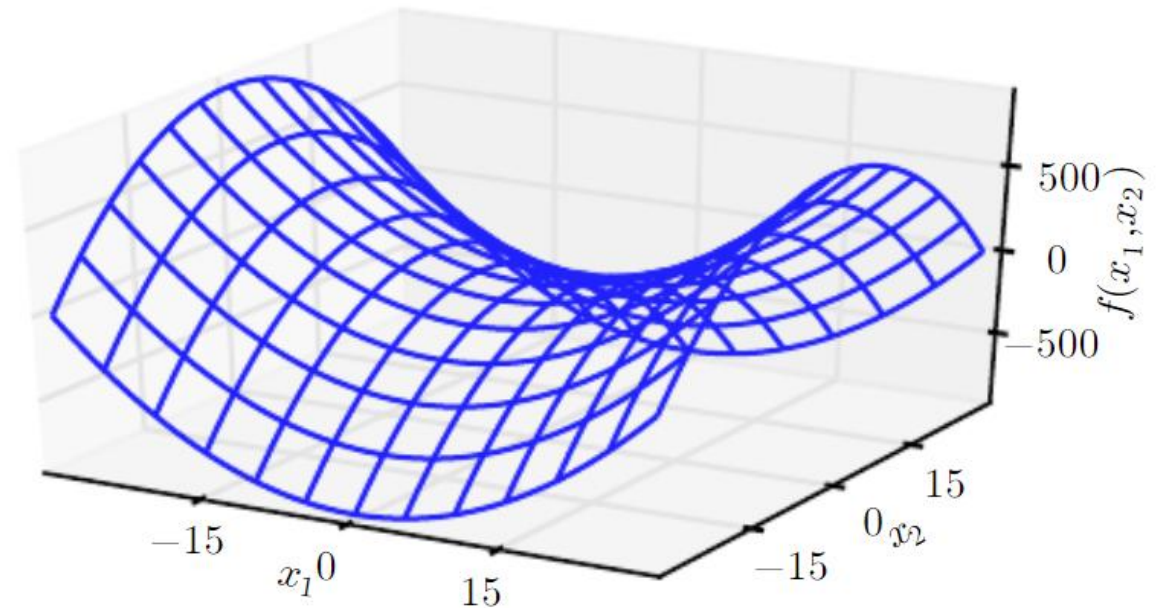
Optimization Challenges – Local Minima

- We can have many local minima (and often do) – many of which may be shallow local minima.
- Can we get past these to find a lower loss?
- Although, for larger models this does not seem to be a major problem
- Check by plotting the norm of the gradient over time – if it doesn't decrease to insignificant size, the problem isn't a local minimum.

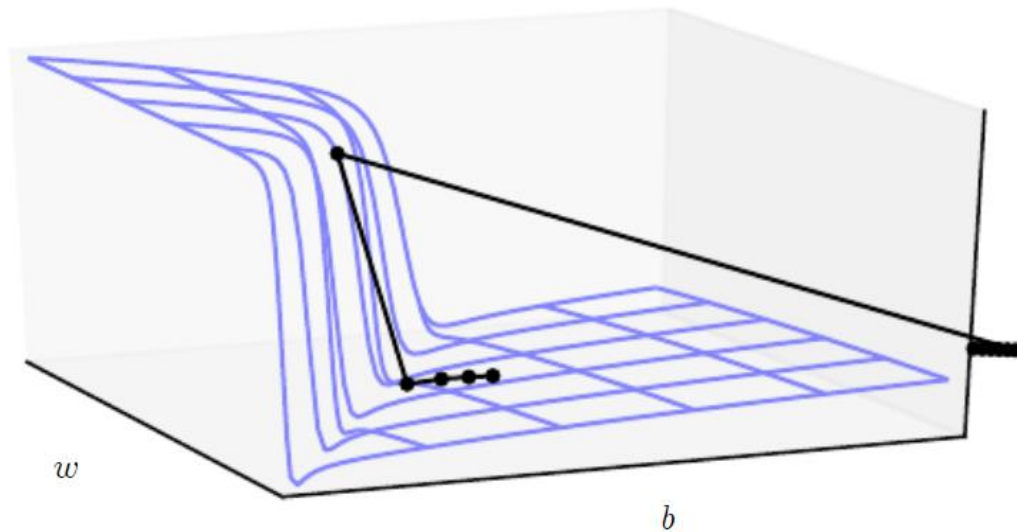


Saddle Points ...

- These are local minima in some directions, and local maxima in others.
- They are very common in high dimensional spaces where the “Hessian matrix” has a mix of positive and negative eigenvalues in n -dimensional space since it is exponentially unlikely that they are all positive (giving a minima).
- Can cause issues since magnitude of gradient is very different in different directions (which is a common theme that causes optimization issues).



Cliffs ...



Cliffs and exploding gradients

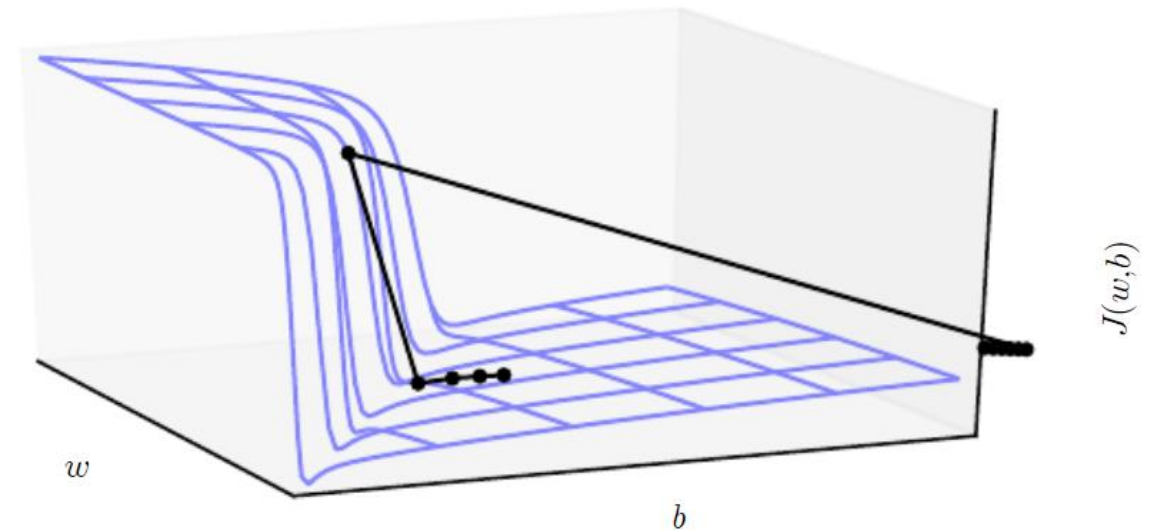
- Some deep networks have very steep regions in the objective function, often from multiplication of large weights together (e.g. in recurrent networks), which leads to high derivatives, so a gradient descent update can catapult parameters far from optimality.
- Clipping gradients are options to fix this.

Inexact noisy gradients and bumpy loss functions

- While algorithms assume a perfect gradient, we usually have a numerical approximation based on sampling-based estimates which can cause issues in some cases.
- Essentially bumps need to be smoothed out !

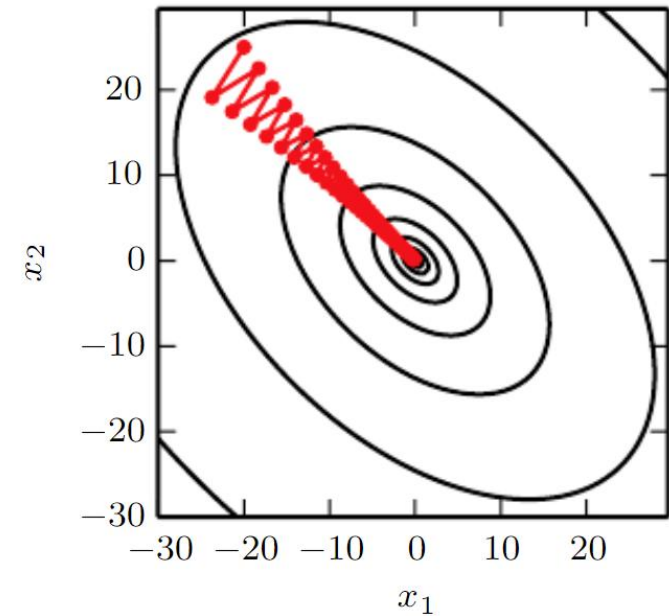
Plateaus and Flat Regions

- Some areas of the loss surface may be very flat compared to other areas.
- These can cause similar issues to vanishing gradients – that Stochastic Gradient Descent becomes ineffective with a fixed learning rate as only very small changes are made to parameter values.
- This causes the algorithm to spend a long time trying to get across flat areas of the loss surface.
- Again, we have the theme of very large gradients (cliffs) mixed with very small gradients causing issues for a fixed learning rate.



Steep Valleys ...

- Steep valleys have large gradients in one direction with small gradients in another.
- These can cause issues with a fixed step size since the algorithm can spend all its time going from one side of the valley to another and not making progress down the shallow gradient to the minimum.

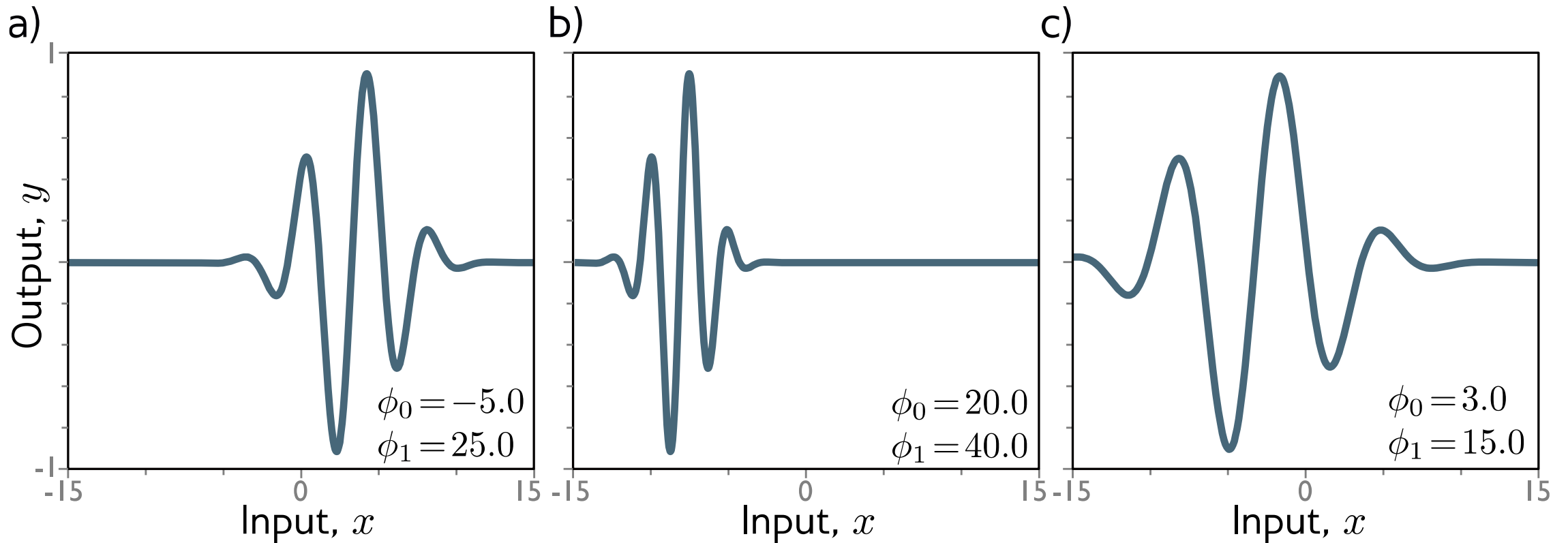


We will introduce two key innovations that can potentially help with these issues –

1. Momentum
2. Adaptive Learning Rate Methods (ADAM)

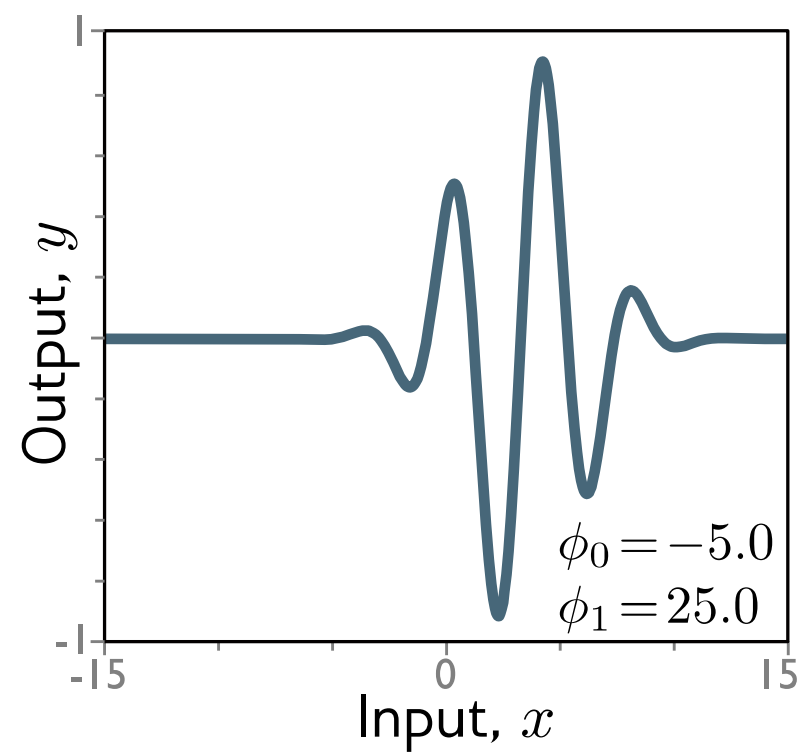
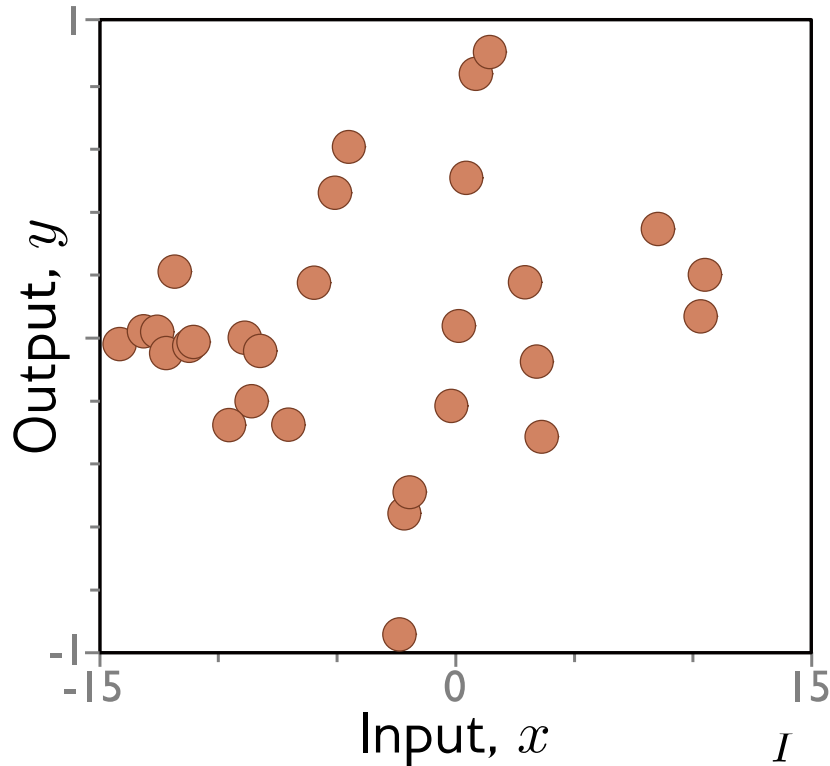
Motivating Example: Gabor Function

$$f[x, \phi] = \sin[\phi_0 + 0.06 \cdot \phi_1 x] \cdot \exp\left(-\frac{(\phi_0 + 0.06 \cdot \phi_1 x)^2}{8.0}\right)$$



Gabor model

$$f[x, \phi] = \sin[\phi_0 + 0.06 \cdot \phi_1 x] \cdot \exp\left(-\frac{(\phi_0 + 0.06 \cdot \phi_1 x)^2}{8.0}\right)$$



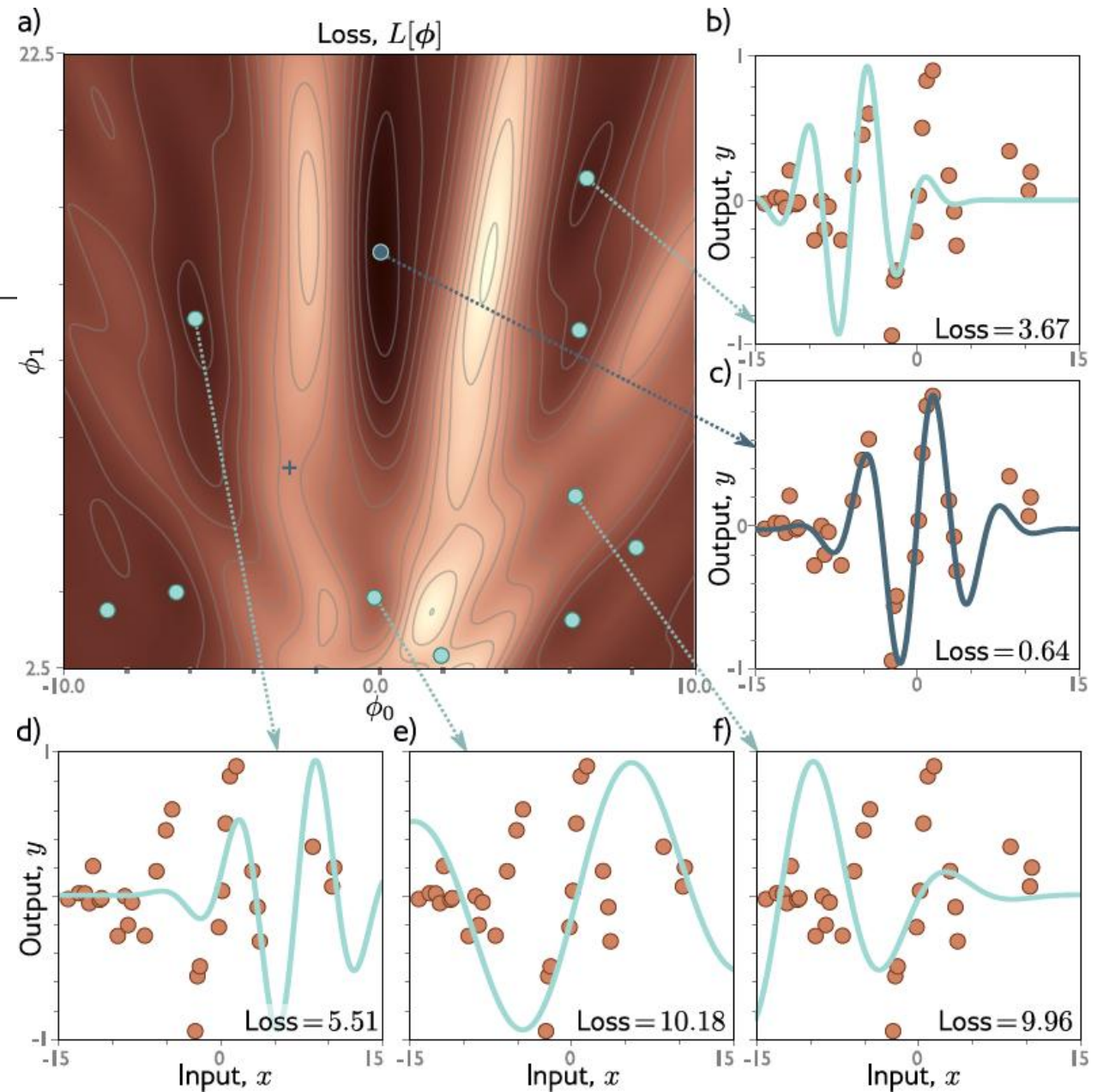
$$L[\phi] = \sum_{i=1}^I (f[x_i, \phi] - y_i)^2$$

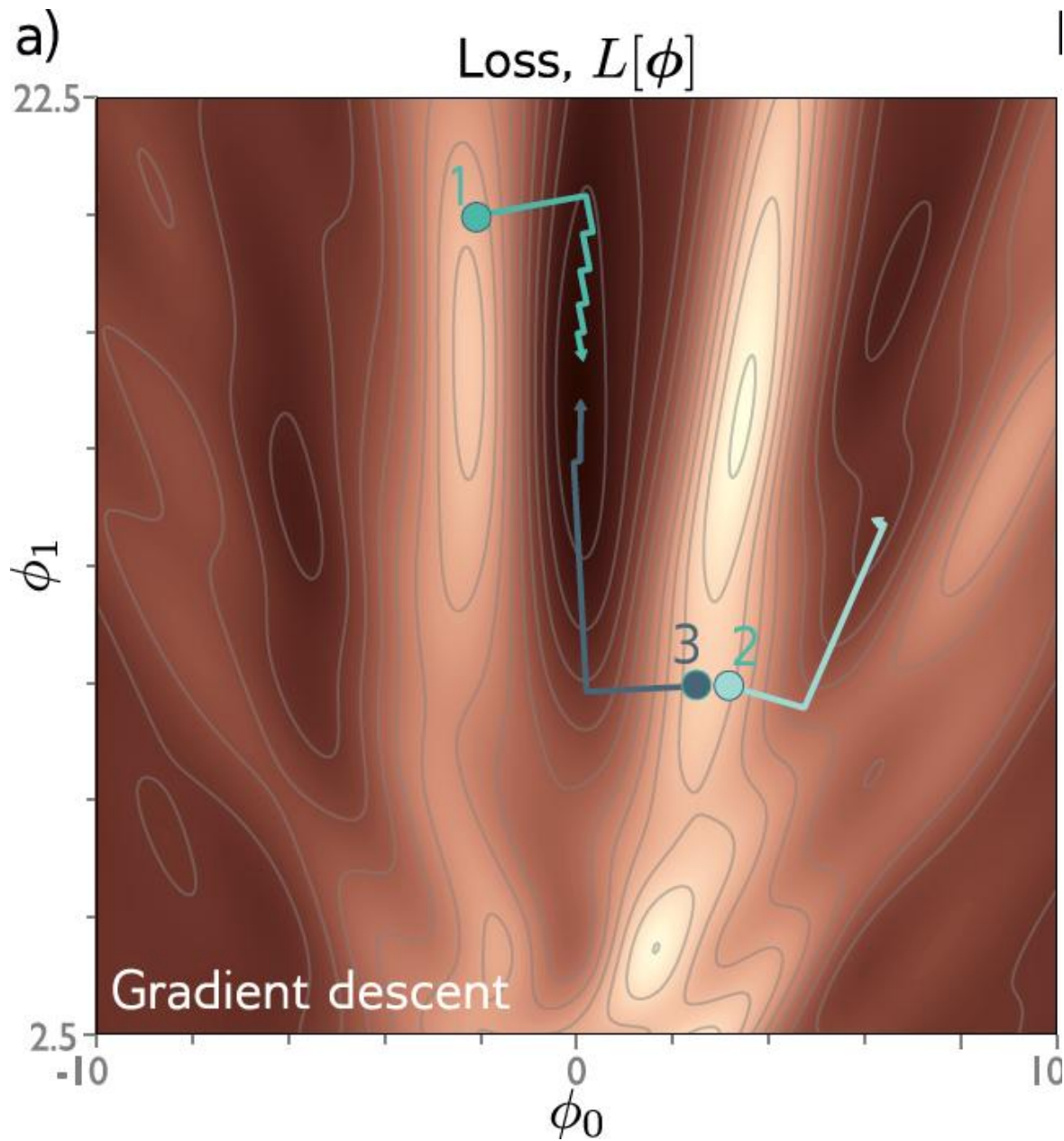
Loss Surface

We want to optimize a Gabor Model to fit a given dataset (orange points).

This shows the loss surface for various choices of parameters.

You can see that dark areas represent valleys with low loss and well-fitting data and light areas are mountains with higher loss values.

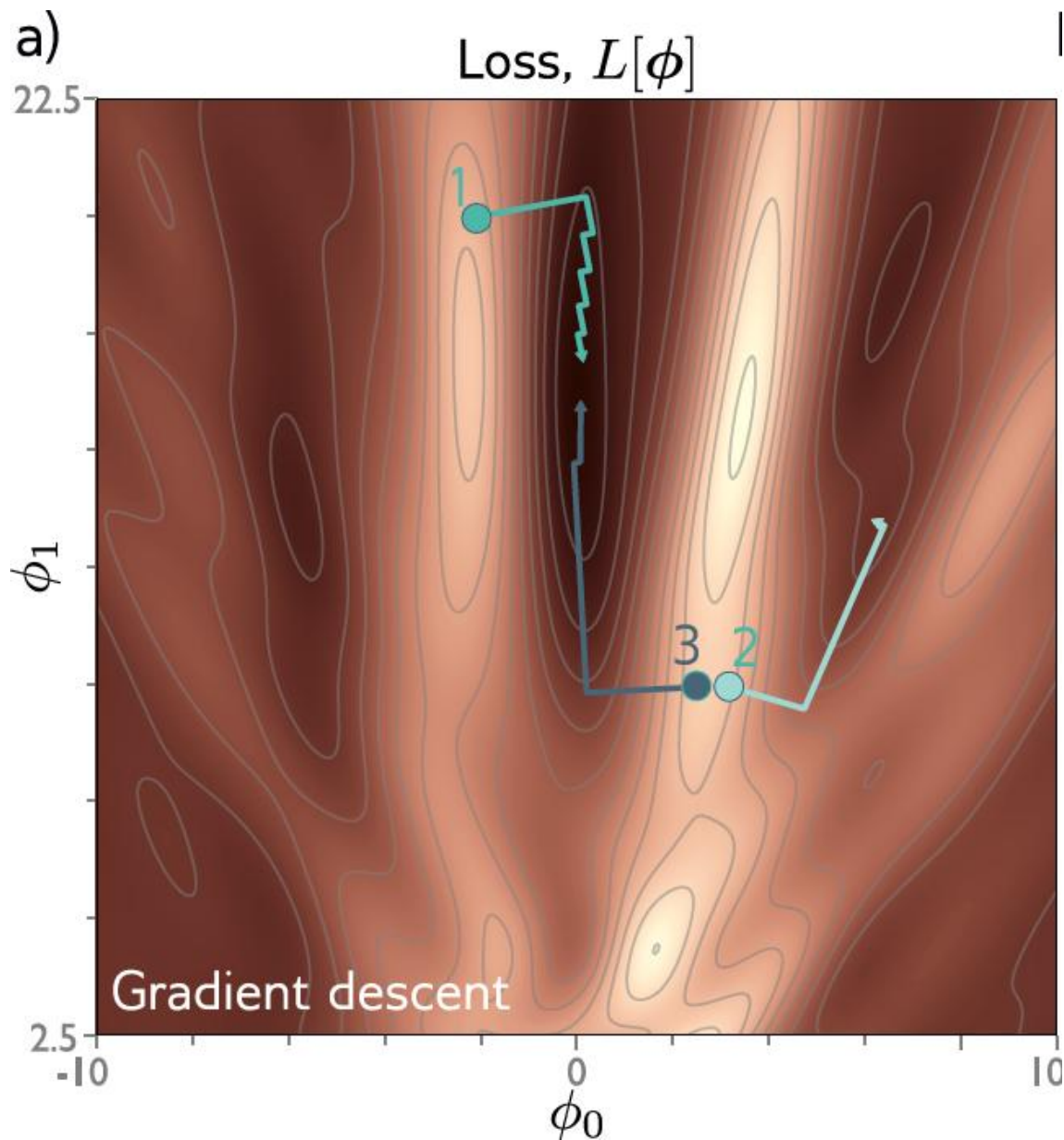




Gradient descent gets to the global minimum if we start in the right “valley”

Otherwise, descent to a local minimum

Or get stuck near a saddle point.



Stochastic gradient descent

Before (full batch descent)

$$\phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i=1}^I \frac{\partial \ell_i[\phi_t]}{\partial \phi},$$

After (SGD)

$$\phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi},$$

Fixed learning rate α

a)

Loss, $L[\phi]$

22.5

 ϕ_1

2.5

-10

 ϕ_0

10

Gradient descent

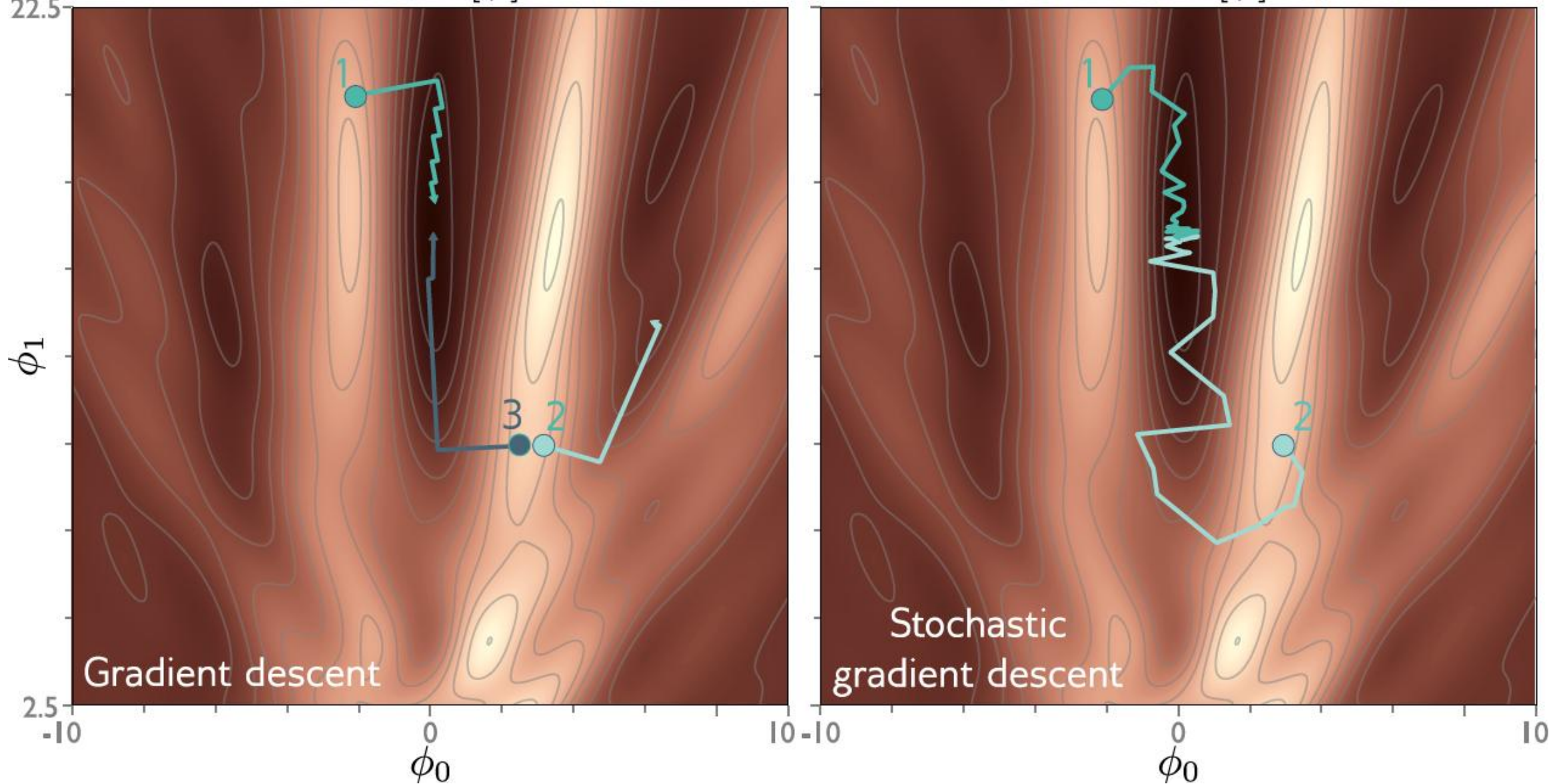
b)

Loss, $L[\phi]$

-10

 ϕ_0

10

Stochastic
gradient descent

Properties of SGD

Can escape from local minima

Adds noise, but still sensible updates as based on part of data

Uses all data equally

Less computationally expensive

Seems to find better solutions

Doesn't converge in traditional sense

Learning rate schedule helps convergence – decrease learning rate over time

Standard SGD Algorithm

Algorithm 8.1 Stochastic gradient descent (SGD) update at training iteration k

Require: Learning rate ϵ_k .

Require: Initial parameter $\boldsymbol{\theta}$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow +\frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), \mathbf{y}^{(i)})$

 Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \hat{\mathbf{g}}$

end while

What size batch to use?

Larger batches give more accurate gradients, but with less than linear returns

GPU-like multicore architectures typically underutilised by small batches, so below some minimum size there is no computational saving of cutting batch size

Memory scales with the batch size, so for larger networks, this is often **the** limiting factor.

Depending on hardware, GPUs often run better on power-of-2 batch sizes. Typical uses will be minibatch sizes from 32 to 256

Small batches can have a regularising effect because of the noise they add to the learning process.

- (might need small learning rate, and will have long learning time computationally).

Improving SGD: Momentum

Designed to accelerate learning, especially with small consistent gradients or noisy gradients

Accumulates exponentially decaying moving average of past gradients (as a velocity).

Inspired from physical interpretation of the optimization process: Imagine you have a small ball rolling on a surface defined by the loss function.

$$v = \alpha v - \epsilon \nabla_{\theta} \left(\frac{1}{m} \sum_i L(f(x^{(i)}; \theta), y^{(i)}) \right),$$

$$\theta = \theta + v,$$

Hyperparameters

$\alpha \in [0, 1)$

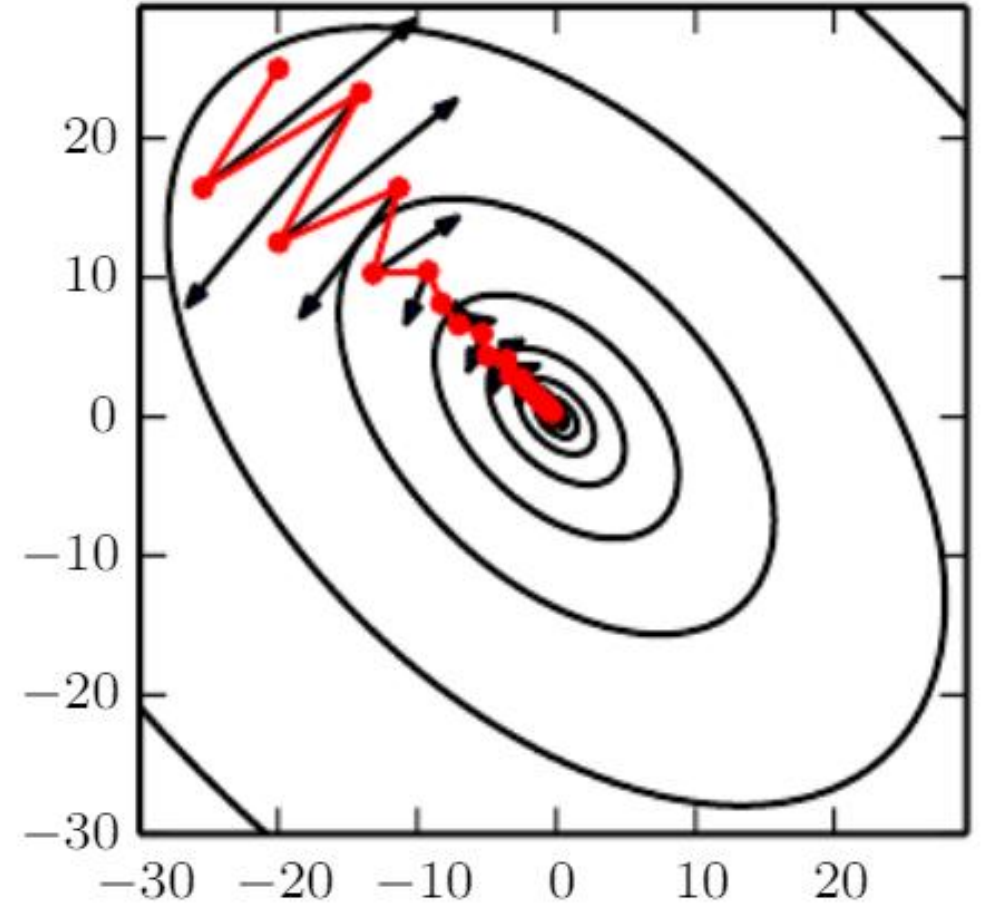
learning rate ϵ

Momentum

Red line shows movement with momentum whereas black arrows show gradient.

Momentum helps:

- ❑ To follow this valley down even though it has problematic steep sides (it is ill-conditioned).
- ❑ Smoothing out the direction of the stochastic gradients.
- ❑ Get past shallow local minimum and flat areas.



SGD Algorithm with momentum

Algorithm 8.2 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ , momentum parameter α

Require: Initial parameter θ , initial velocity v

while stopping criterion not met **do**

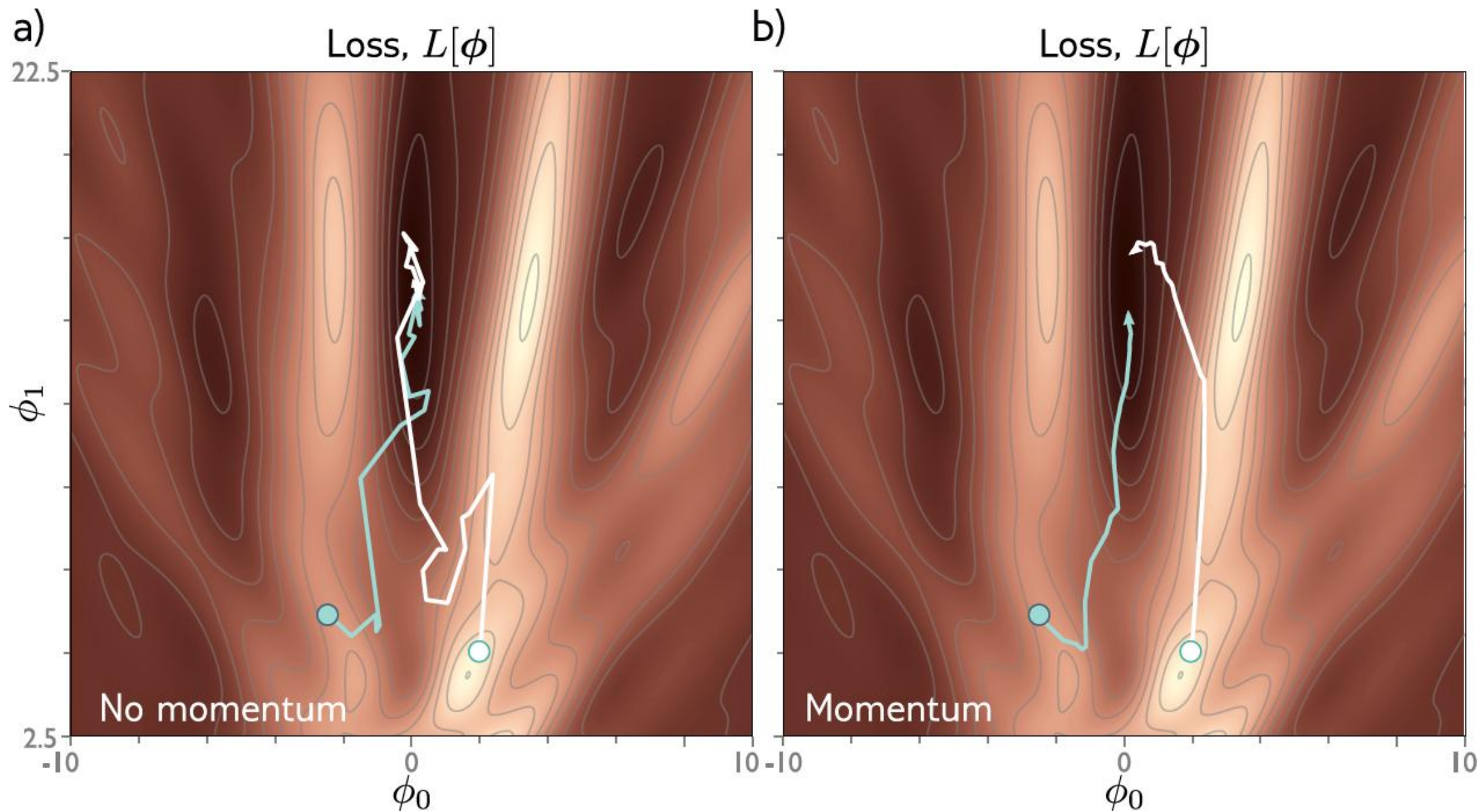
 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$.

 Compute velocity update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \mathbf{g}$.

 Apply update: $\theta \leftarrow \theta + \mathbf{v}$.

end while



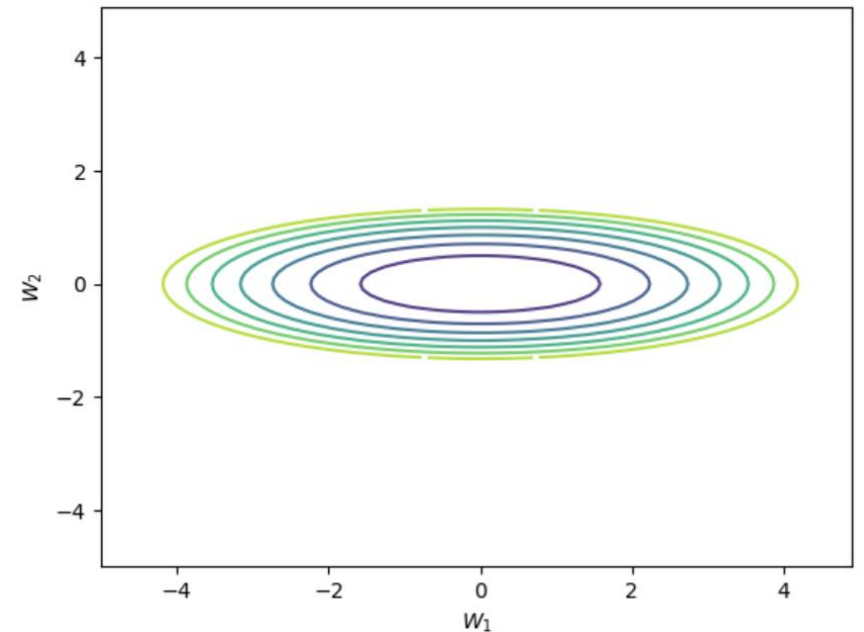
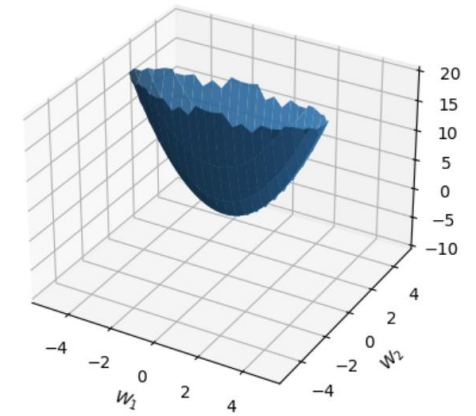
Adaptive Learning Rates

The learning rate turns out to be one of the most Important hyper-parameters for fast and successful optimization of a deep neural network.

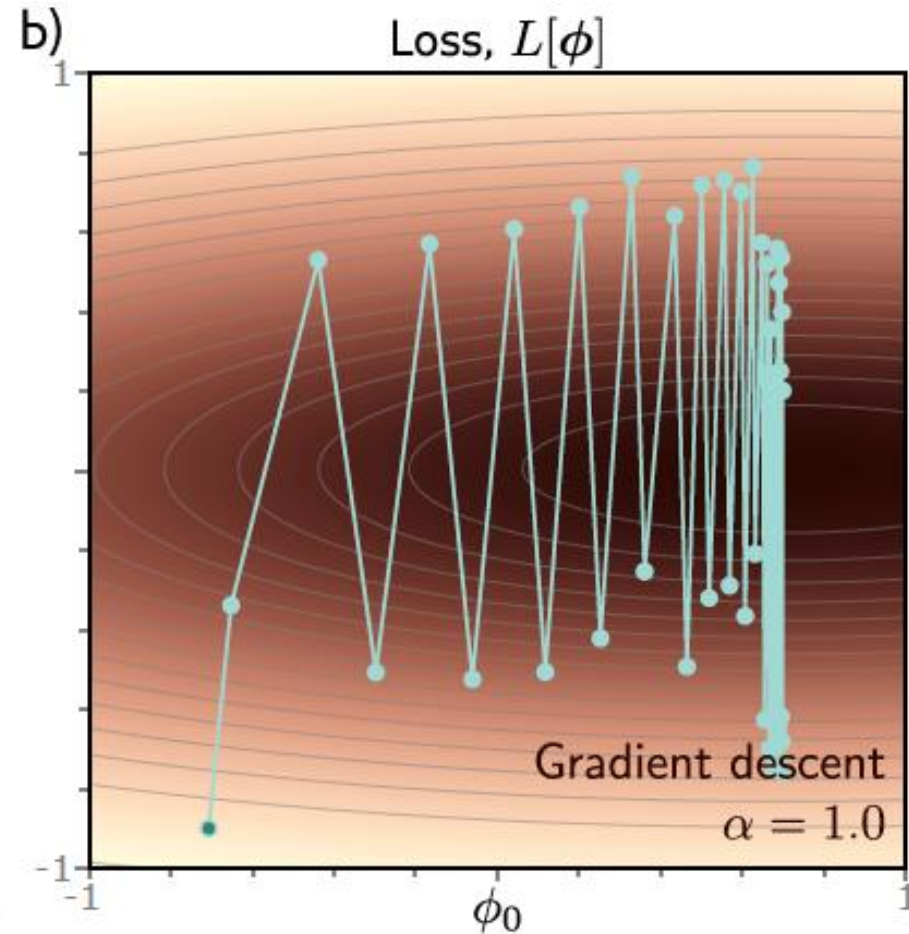
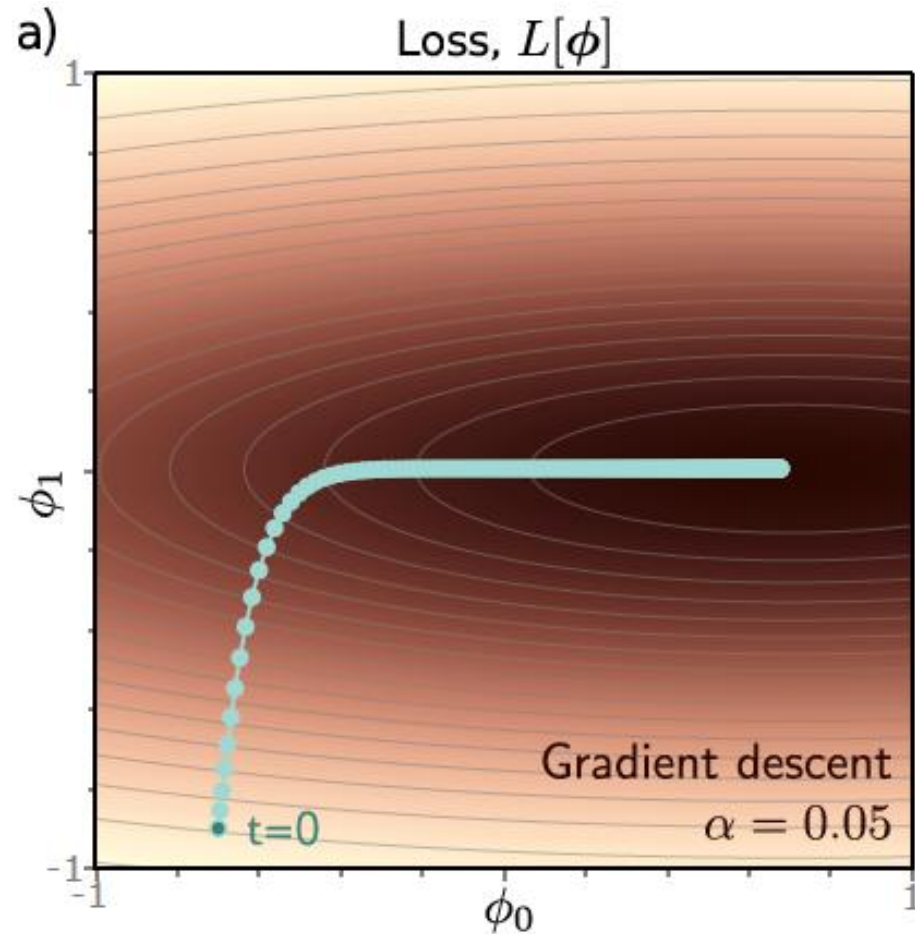
A key issue is when one parameter has a steep loss gradient and another a shallow loss gradient (like this example).

You want to take big learning rate steps when the gradient is shallow but much smaller steps when the gradient is steep !

There are many adaptive learning rate algorithms but we will look at a couple of popular ones called RMSprop and ADAM.



Steep valley issues with gradient descent: learning rate is either too small or too big



Adaptive Learning Rate: RMSprop

The step size for individual parameters is divided by the accumulated squared gradient for that parameter.

Parameters with consistently large gradients take small steps and those with consistently small gradients take larger steps.

Algorithm 8.5 The RMSProp algorithm

Require: Global learning rate ϵ , decay rate ρ

Require: Initial parameter θ

Require: Small constant δ , usually 10^{-6} , used to stabilize division by small numbers

Initialize accumulation variables $\mathbf{r} = 0$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$.

 Accumulate squared gradient: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$.

 Compute parameter update: $\Delta \theta = -\frac{\epsilon}{\sqrt{\delta + \mathbf{r}}} \odot \mathbf{g}$. ($\frac{1}{\sqrt{\delta + \mathbf{r}}}$ applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta \theta$.

end while

ADAM (ADaptive Moments)

Variant of RMSProp + momentum with a few important distinctions

1. Momentum is incorporated directly via an estimate of the first order moment of the gradient (essentially summing the gradients with exponential decay weighting like the velocity before).
2. Adaptive weighting is incorporated via an estimate of the second order moment of the gradient (essentially summing the gradient squares for individual parameters similar to RMSprop).

To date, ADAM has largely become one of the most popular optimization algorithm for training deep learning systems, partly because it is quite robust to a wide range of hyperparameter values.

ADAM

Incorporates momentum via the accumulated first moment (gradient terms).

Does adaptive weighting of parameters via the accumulated second moment (gradient squared terms).

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$. (Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ used for numerical stabilization (Suggested default: 10^{-8})

Require: Initial parameters θ

Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Initialize time step $t = 0$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

$t \leftarrow t + 1$

Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$

Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$

Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$

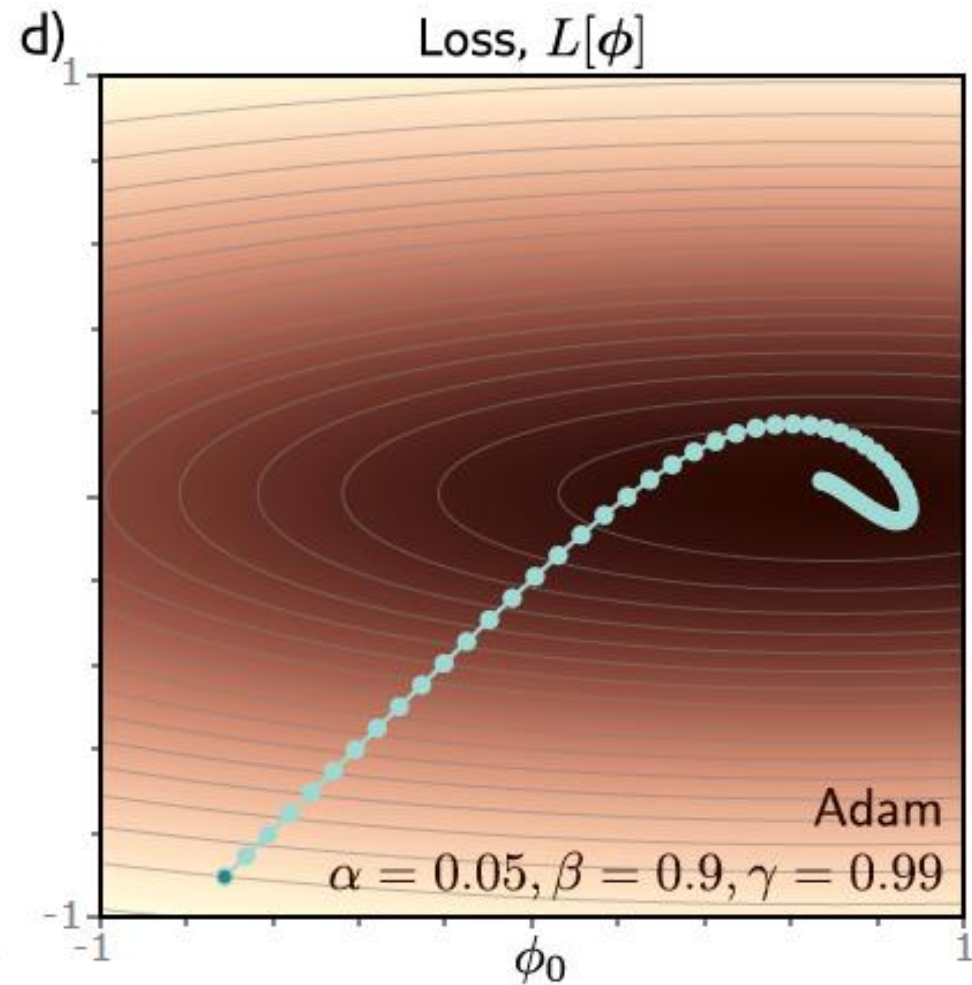
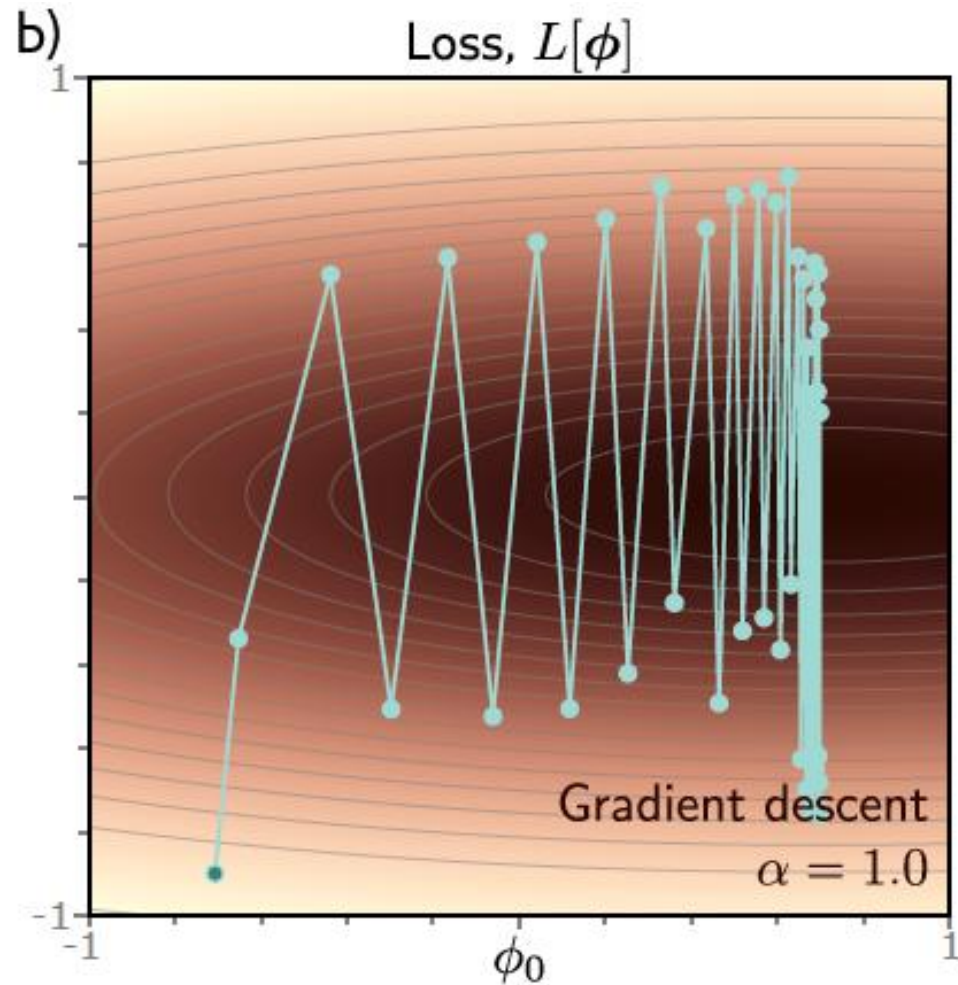
Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$

Compute update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (operations applied element-wise)

Apply update: $\theta \leftarrow \theta + \Delta \theta$

end while

Adaptive moment estimation (Adam)



Parameter initialisation

- ❑ Good initialization of parameters can strongly affect performance (speed of learning and final generalisation error), and whether the network converges at all.
- ❑ An initialisation strategy tries to achieve some 'nice' property to start with, but do these get maintained during learning?
- ❑ Most basic is 'symmetry breaking'. What would happen if all weights were initialised identically? Hence random initialization is important.
- ❑ Also making weights too large can lead to saturation of neurons or explosion of forward values or gradients.
- ❑ Typically use small random weights from Gaussian or uniform distributions.

Example initialisation heuristics

torch.nn.Linear Layer API

- **weight** (*torch.Tensor*) – the learnable weights of the module of shape (out_features, in_features). The values are initialized from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$, where $k = \frac{1}{\text{in_features}}$
- **bias** – the learnable bias of the module of shape (out_features). If **bias** is **True**, the values are initialized from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ where $k = \frac{1}{\text{in_features}}$

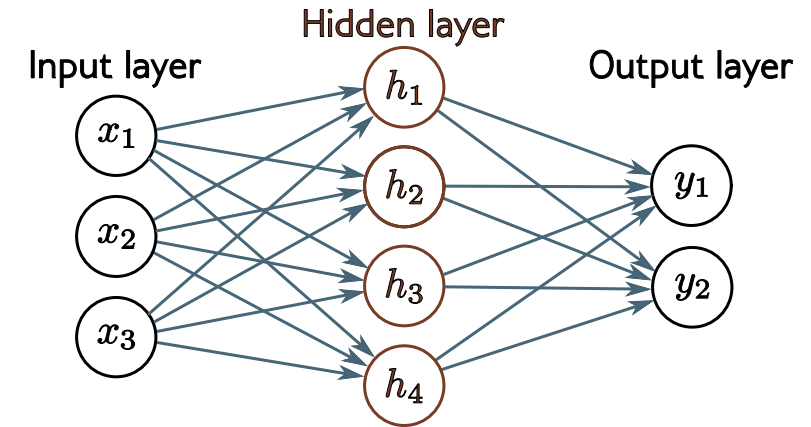
- Sample from $U(-\frac{1}{\sqrt{m}}, \frac{1}{\sqrt{m}})$ for m inputs, n outputs.
- (Glorot & Bengio 2010) suggest normalised initialisation

Xavier (or Glorot) Uniform
Initialization for sigmoid / tanh

$$W_{I,j} \sim U \left(-\sqrt{\frac{6}{m+n}}, \sqrt{\frac{6}{m+n}} \right).$$

to compromise with the goal of initialising all layers to have same activation variance and the same gradient variance. (it is based on an approximation of a series of matrix multiplications with no nonlinearities – obviously violated by typical networks).

For this layer:
 $m = 3$ inputs
 $n = 4$ output



One of the key properties is to obtain stable gradients

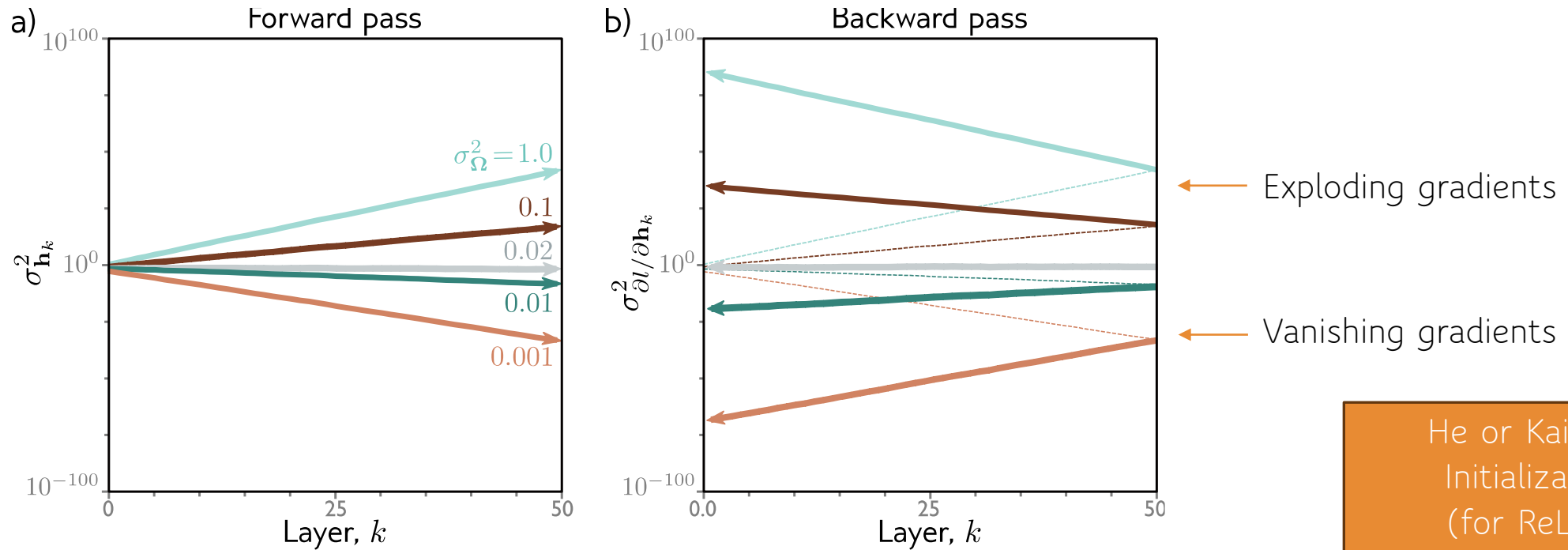


Figure 7.4 Weight initialization. Consider a deep network with 50 hidden layers and $D_h = 100$ hidden units per layer. The network has a 100 dimensional input $\mathbf{x}$ initialized with values from a standard normal distribution, a single output fixed at $y = 0$, and a least squares loss function. The bias vectors β_k are initialized to zero and the weight matrices Ω_k are initialized with a normal distribution with mean zero and five different variances $\sigma_{\Omega}^2 \in \{0.001, 0.01, 0.02, 0.1, 1.0\}$. a)

He or Kaiming
Initialization
(for ReLUs)
See: `torch.nn.init`

$$\sigma_{\Omega}^2 = \frac{2}{D_h} = \frac{2}{100} = 0.02$$

Summary

Parameter optimization is an important topic for deep learning – all these techniques help to speed up or make optimization more robust by handling difficult aspects of the loss surface:

1. SGD minibatch size
2. Momentum
3. Adaptive Learning Rate Methods (RMSprop and ADAM)
4. Parameter initialization

Background Reading

Understanding Deep Learning (Simon Prince) Chapter 6 (Fitting Models) and the last part of Chapter 7 provide more detail about this material.

Relevant sections of Chapter 4 and Chapter 8 of Goodfellow et al 2018

- <https://www.deeplearningbook.org/contents/optimization.html>